9주차 · 라이다 장애물회피, 파이썬 유도와 시뮬링크 제어

캡스톤디자인 2026-2 · 충남대학교 자율운항시스템공학과 | 갱신 2026-10-27

★ 이번 학기 녹화 강의 영상은 여기에 올라감 — 수업에서 놓친 부분은 영상으로 확인할 것

https://youtube.com/playlist?list=PLK_f1-krzJG8

- 매주 수업 뒤 그 주차 영상이 이 재생목록에 추가됨
- 아래 "참조 강의" 는 **지난 학기까지의 선수 과목**이고, 이 재생목록은 **이번 학기 본 수업** 녹화임

★ 참조 강의 — 본 과목이 전제하는 배경

아래 다섯 과목은 **본 과목 담당 교수가 직접 강의한 것**이며, 본 과목이 전제하는 배경 지식에 해당함. 학부 기초에서 대학원 과정까지 이어지므로 부족한 지점부터 시작하면 됨. 본 문서에서 쓰는 좌표계·기호·유도 과정은 아래 강의에서 상세히 다루므로, 선수 지식이 부족한 경우 먼저 보고 돌아올 것.

#	과목	수준	언어	영상	자료·코드
1	**제어공학** — 전달함수, 되먹임, 안정도, 근궤적, PID. 모든 것의 토대	학부	한국어	재생목록	드라이브
2	**제어시스템설계** — 해석이 아니라 설계. 사양 결정, 루프 정형화, 이산 구현	학부	한국어	재생목록	드라이브
3	**캡스톤디자인** — 본 과목의 지난 학기 강의. 이동체 프로젝트를 처음부터 끝까지	학부	한국어	재생목록	드라이브
4	**제어공학특론** — 좌표계, 6자유도 운동방정식, 회전행렬과 오일러각, 선형화와 트림	대학원	영어	재생목록	드라이브
5	**센서신호처리 및 융합** — 센서 모델, 잡음, 추정, 다중센서 융합	대학원	영어	재생목록	드라이브

MATLAB·Simulink 가 처음이라면 4주차 실습 전에 아래를 끝낼 것. 본 과목의 제어기 실습은 전부 Simulink 로 진행함. Onramp 는 무료이며 각각 몇 시간이면 끝남

도구	시작 지점
MATLAB	MATLAB Onramp · Core MATLAB Skills
Simulink	Simulink Onramp · 담당 교수 Simulink 강의 1부 · 2부

★ 강의자료 저장소 — 처음 한 번만 `git clone`, 그 뒤로는 `git pull`

저장소	주소	받는 자리 (WSL)
강의자료 — 주차 문서 · Simulink 모델 · MATLAB 스크립트	**https://github.com/wkyouncnu/Capstone-Design**	`~/Capstone-Design`
ROS 2 예제 패키지 `usv_basics` — 2주차부터 쓰는 노드 코드	**https://github.com/wkyouncnu/usv_basics**	`~/capstone_ws/src/usv_basics`

언제	명령 (VS Code 의 WSL 창 터미널)	하는 일
처음 한 번	`cd ~ && git clone https://github.com/wkyouncnu/Capstone-Design.git`	강의자료 전체를 받음 (약 770 MB · 2026-10-16 실측 1분 40초)
처음 한 번	`mkdir -p ~/capstone_ws/src && cd ~/capstone_ws/src && git clone https://github.com/wkyouncnu/usv_basics.git`	예제 패키지를 받음. 이어서 `cd ~/capstone_ws && colcon build --symlink-install`
매주 수업 전	`cd ~/Capstone-Design && git pull` · `cd ~/capstone_ws/src/usv_basics && git pull`	**바뀐 파일만** 받음. 다시 clone 하지 않음
무엇이 바뀌었는지	`git log --oneline -10` · `git show --stat HEAD`	교수가 갱신한 내역을 확인

- 두 저장소 모두 공개 — 로그인 없이 받아짐
- 주차 문서는 `~/Capstone-Design/1_2026-2학기_강의자료/10-주차별-강의자료/` 에 있음. MATLAB 은 같은 폴더를 `\\wsl.localhost\Ubuntu-22.04\home\<사용자명>\Capstone-Design\...` 로 엶
- `usv_basics` 를 받은 뒤 새 노드가 생겼으면 `colcon build --symlink-install` 을 한 번 더 돌림
- 본인이 고친 파일 때문에 `git pull` 이 멈추면 `git stash` 로 치워 두고 다시 받음 → 강의자료는-한-번-받고-git-pull-로-갱신한다

- **과목**: 캡스톤디자인 (2026-2) · 충남대학교 자율운항시스템공학과
- **이번 주차 학습 내용**: 배에 **3D 라이다**를 달아 앞을 보게 하고, 목표까지의 통로가 막혔을 때만 **파이썬이 헤딩 명령을 내도록** 함. 제어기는 5주차 것을 **한 줄도 바꾸지 않음**

★ 9주차는 중간 발표 주간임

- 중간고사는 없음. 대신 이번 주차에 **중간 발표**를 함 — 전체 평가의 **15%**
- 가져올 것 둘: ① 1~8주차 결과물 중 하나의 **동작 데모**, ② 팀별 Term Project **제안서 초안**
- 발표 순서와 시간 배분은 평가와-팀운영, 제안서 양식은 Term-Project-명세 를 따름
- 이번 주차 실습은 발표 뒤에 진행함. 실습 분량이 많으므로 **① 터미널 1 을 발표 전에 미리 띄워 둘 것**

★ 시작 전 확인

- **강의자료부터 갱신**: VS Code WSL 창 터미널에서 `cd ~/Capstone-Design && git pull` (강의자료는-한-번-받고-git-pull-로-갱신한다)
- 6주차 `W06_vrx_lite/run_vrx_bow.sh` 가 동작했어야 함 — 이번 주차의 배는 **그 선수추진기 배에 라이다만 더 단 것**임
- 5주차 `W05_1_vrx.slx` 가 동작했어야 함 — 이번 주차 모델은 **그것을 복사해서** 만듦
- MATLAB **R2024b + Simulink + ROS Toolbox**
- 배포 폴더 `W09_simulink/` 와 `W09_vrx_lite/`
- WSL 메모리 여유 **8 GB 이상** — 라이다를 달면 Gazebo 서버가 장면을 렌더함

★ 이번 주차 실행 요약 — 네 덩어리만 치면 됨

자세한 설명은 2부. 아래 순서는 2026-10-27 기준으로 확인한 것임

① 터미널 1 — 라이다를 단 Gazebo 띄우기 (창이 뜨면 닫지 않고 둠)

```
cd ~/Capstone-Design && git pull
cd ~/Capstone-Design/1_2026-2학기_강의자료/10-주차별-강의자료/W09_vrx_lite
bash ../W03_vrx_lite/stop_vrx.sh
bash run_vrx_ca.sh
```

② 터미널 2 — 회피 쪽 노드 셋 띄우기 (브리지 · 회피기 · 극좌표 그림)

```
cd ~/Capstone-Design/1_2026-2학기_강의자료/10-주차별-강의자료/W09_vrx_lite
bash run_ca_nodes.sh
```

③ MATLAB — 이음새를 붙여 돌리기

```
cd('\\wsl.localhost\Ubuntu-22.04\home\<사용자명>\Capstone-Design\1_2026-2학기_강의자료\10-주차별-강의자료\W09_simulink')
W09_setup
W09_vrx_run
```

④ 끝낼 때 — 반드시 정리 (남은 `gz sim` 이 메모리를 수십 GB 까지 먹음)

```
bash ../W03_vrx_lite/stop_vrx.sh
```

이럴 때	할 것
RTF 가 0.3 아래로 떨어짐	① 을 `bash run_vrx_ca.sh ultralite` 로 다시 (1-3 절)
Rviz 로 장애물을 눈으로 보고 싶음	터미널 3 에서 `rviz2 -d w09_ca.rviz` (실습 D)
시뮬링크 없이 회피만 먼저	② 를 `GOAL="-532 312" DEMO_THRUST=1 bash run_ca_nodes.sh` 로 (실습 F)
라이다가 무엇을 보는지 숫자로	`bash probe_lidar.sh` · `bash check_obstacles.sh` (실습 B · D)
Gazebo 창이 안 뜸	Windows PowerShell 에서 `wsl --shutdown` → 터미널을 새로 열고 ① 부터
다음 실험	④ 부터 다시

> **8주차까지는 아는 길만 갔음**
>
> - 5주차 LOS 유도는 "다음 웨이포인트가 저기 있으니 그쪽으로 선수를 돌려라" 가 전부였음 — 길에 무엇이 있는지는 보지 않았음
> - 8주차 Stateflow 는 **어느 모드로 갈지**를 골랐지만, 모드 안에서 가는 길은 역시 미리 정해진 선이었음
> - 즉 지금까지의 배는 **눈이 없었음**. 센서는 자기 위치와 자세만 알려 주었음
> - 이번 주차에 눈(라이다)을 달고, 눈이 "앞이 막혔다" 고 할 때만 길을 바꿈
> - 바꾸는 것은 **길뿐**임. 그 길을 따라가는 제어기는 5주차 것을 그대로 씀

이 주차를 마치면 할 수 있어야 하는 것

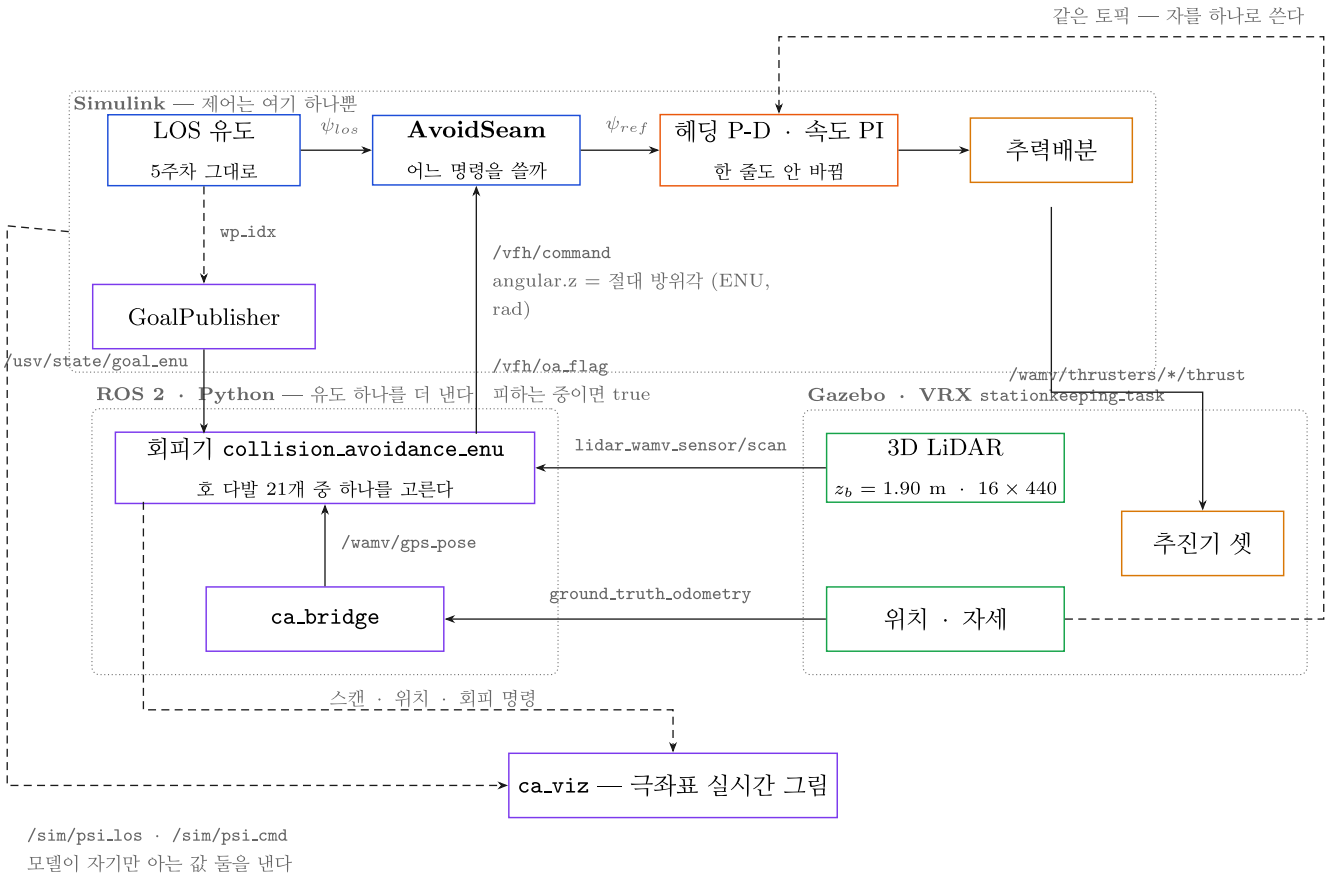
1. **유도와 제어의 경계**를 말하고, 회피가 왜 유도단에 들어가는지 설명하기
2. VRX WAM-V 에 **3D 라이다를 달고** 위치(특히 높이)를 **실측 근거로** 정하기
3. ROS 의 **FLU** 축과 본 과목의 **FRD** 축을 변환하고, 부호를 틀리면 무엇이 깨지는지 말하기
4. 라이다 성능을 세 판으로 낮춰 보고, **RTF 와 발행 주기 중 무엇이 실제 제약인지** 측정으로 가리기
5. Rviz2 에서 `PointCloud2` 로 장애물을 띄우고, **다가갈수록 거리가 줄어드는 것**을 숫자로 확인하기
6. 라이다 거리값을 **장애물 점 묶음**으로 바꾸는 전처리(걸러내기·군집)를 설명하기
7. **호 다발(arc bank)** 회피기의 세 걸음 — 깔기 · 지우기 · 점수 매기기 — 를 코드와 대조하며 설명하기
8. 선체를 **원이 아니라 직사각형**으로 재는 이유를 숫자로 제시하기
9. `oa_flag` 가 **거리 문턱이 아니라 통로 판정**임을 설명하고, 그래서 언제 켜지는지 예로 들기
10. Simulink 쪽 **이음새(스위치 + 명령 나이)** 를 만들고, 명령이 끊겼을 때 무엇으로 되돌아가는지 보이기

준비물

구분	내용
소프트웨어	MATLAB R2024b · Simulink · ROS Toolbox · WSL2 Ubuntu 22.04 · ROS 2 Humble · Gazebo Garden · VRX 2.4.1 · `rviz2` · `python3-matplotlib`
배포 폴더	`W09_vrx_lite/` (URDF 생성·실행·측정·시각화 11개) · `W09_simulink/` (모델 1개 + m 파일 4개)
선행	6주차 (선수추진기 배와 `run_vrx_bow.sh`) · 5주차 (LOS 유도와 `W05_1_vrx.slx`) · 3주차 (VRX lite 실행과 좌표계) · 2주차 (토픽)
시간	이론 50분 + 실습 150분 (중간 발표와 같은 주차이므로 발표 뒤 실습)

1부 · 이론

1-1. 역할 분담 — 유도는 둘, 제어는 하나



이 그림에서 5주차와 달라진 자리는 AvoidSeam 한 칸뿐이다. 헤딩 P-D · 속도 PI · 추력배분 · 운동모델은 블록 하나도 바뀌지 않았다. 유도가 둘(LOS 와 회피)로 늘고, 그 둘 중 하나를 고르는 상자가 생긴 것이 전부다

- 그림 읽는 법

요소	의미
위쪽 묶음 (Simulink)	제어는 **여기 하나뿐**임. LOS 유도 → AvoidSeam → 헤딩 P-D·속도 PI → 추력배분
파란 상자 `AvoidSeam`	이번 주차에 **새로 생긴 유일한 칸**. 하는 일은 "어느 헤딩 명령을 쓸까" 의 판단 하나
주황 상자 헤딩 P-D · 속도 PI	5주차 것 **그대로**. 블록 하나도 바뀌지 않음
왼쪽 아래 묶음 (ROS 2 · Python)	유도를 **하나 더** 냄. 회피기와 좌표 브리지 둘
오른쪽 아래 묶음 (Gazebo · VRX)	라이다 · 위치·자세 · 추진기 셋
화살표에 붙은 글자	ROS 2 토픽 이름. 두 세계를 잇는 것은 **토픽 다섯**뿐임
점선 화살표	되먹임 — 위치·자세는 Simulink 와 파이썬이 **같은 토픽**에서 받음
맨 아래 `ca_viz`	극좌표 실시간 그림. 파이썬이 그림 (1-9 절)

- 이 그림이 말하는 것은 한 줄임 — **5주차와 달라진 자리는 `AvoidSeam` 한 칸뿐임**
- 유도가 둘(LOS 와 회피)로 늘고, 그 둘 중 하나를 고르는 상자가 생긴 것이 전부임
- 헤딩 P-D · 속도 PI · 추력배분 · 운동모델은 **그대로**임

두 세계를 잇는 토픽 다섯

토픽	메시지형	방향	담는 것
`/wamv/sensors/lidars/lidar_wamv_sensor/scan`	`sensor_msgs/LaserScan`	Gazebo → Python	라이다 거리값 한 고리
`/wamv/sensors/position/ground_truth_odometry`	`nav_msgs/Odometry`	Gazebo → 양쪽	위치·자세 (ENU)
`/usv/state/goal_enu`	`geometry_msgs/Point`	Simulink → Python	목표점. `x` = 동, `y` = 북
`/vfh/command`	`geometry_msgs/Twist`	Python → Simulink	`angular.z` = **절대 방위각**(ENU, rad), `linear.x` = 속력
`/vfh/oa_flag`	`std_msgs/Bool`	Python → Simulink	피하는 중이면 `true`

> ★ `/vfh/command` 의 `angular.z` 는 **각속도가 아님**
>
> - `Twist` 라는 이름 때문에 `angular.z` 를 요 각속도로 읽기 쉬움. 여기서는 **절대 방위각**임
> - 즉 "지금부터 초당 몇 도씩 돌아라" 가 아니라 "**동쪽을 0 으로 재서 이 방향을 보라**" 임
> - 메시지형을 새로 만들지 않고 기존 `Twist` 에 얹은 것이라 이렇게 됨 — 실무에서 흔한 타협이며, 받는 쪽이 뜻을 알고 있어야 함
> - Simulink 쪽은 이 값을 받아 바로 NED 로 돌려 헤딩 명령으로 씀 (1-7 절)

왜 제어까지 파이썬으로 옮기지 않는가

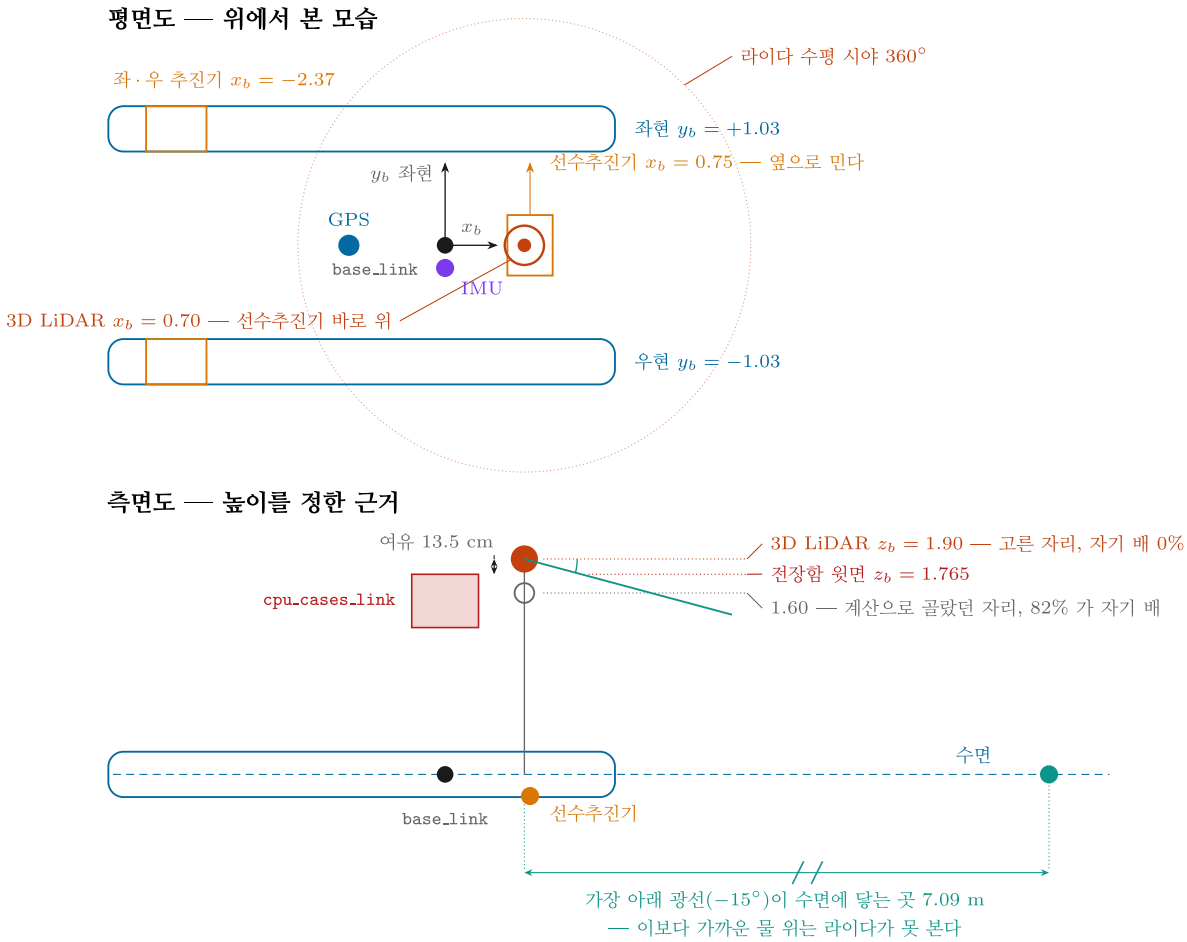
- 연구실 원본 런치(`vrx_avoidance_enu.launch.py`)는 **헤딩 제어까지 파이썬**이 했음. 그쪽이 코드는 짧음
- 그런데 그렇게 하면 이 과목이 1~8주차에 만든 것이 **전부 쓰이지 않음**
- 더 중요한 이유 둘

까닭	설명
제어는 주기가 생명임	헤딩 P-D 는 `Ts_ctrl` = 0.05 s 로 돎. 파이썬 노드는 OS 스케줄러에 맡겨진 20 Hz 임 — 흔들림이 큼
책임이 섞이면 못 고침	배가 휘청일 때 "제어 이득이 나쁜가, 회피 명령이 튀는가" 를 가릴 수 없음. 둘을 갈라 두면 로그 한 장으로 가려짐

- 그래서 경계를 이렇게 그음

하는 일	누가	까닭
어디로 갈까 (유도)	LOS 는 Simulink, 회피는 Python	회피는 센서 자료가 크고 알고리즘이 자주 바뀜 — 스크립트 쪽이 유리함
어느 명령을 쓸까 (판단)	Simulink (`AvoidSeam`)	제어기 바로 앞이므로 제어기와 같은 주기로 돌아야 함
어떻게 갈까 (제어)	Simulink	주기가 일정해야 하고, 이미 튜닝이 끝났음

1-2. 라이다를 어디에 달 것인가 — 높이는 계산이 아니라 실측으로



높이를 **계산으로 1.60 m** 라 정했다가 실측에서 틀린 것을 알았다 — 들어온 값의 82 % 가 장애물이 아니라 자기 배였다. 1.75 → 1.80 사이에서 82 % 가 0 % 로 뚝 떨어진다. 파도에 기우는 것(여유 13.5 cm ≈ 횡동요 18°)까지 보아 **1.90 m** 로 둔다. 값은 비싸지 않다 — 부표를 처음 보는 거리가 4.9 → 5.2 m 로 느는 것뿐이다. 이 그림의 y_b 는 **좌현이 양(+)** 이다 — 5주차 이후 NED 수식과 부호가 반대다

- 그림 읽는 법

요소	의미
위쪽 (평면도)	위에서 본 배. 큰 원은 라이다의 360° 시야
두 개의 긴 사각형	좌·우 선체(hull). 가운데가 비어 있는 쌍동선임
이중 원	라이다. `base_link` 에서 앞으로 **0.70** m, 좌우 중앙
작은 사각형 셋	추진기 셋 — 좌·우·선수 (6주차에서 단 것)
아래쪽 (측면도)	옆에서 본 배. 가로 실선은 수면
붉은 반투명 상자	`cpu_cases_link` — 라이다 바로 아래에 있는 장비함. 윗면이 수면 기준 **1.765** m
세 개의 높이 눈금	시험해 본 높이들. 채택한 **1.90** m 와 그 아래 두 개
13.5 cm 치수선	장비함 윗면과 라이다 사이 여유
아래로 기운 가는 선	가장 아래 광선(**−15°**). 수면과 **7.09** m 앞에서 만남

> ⚠ **ROS 의 축과 본 과목의 축은 y 와 z 의 부호가 반대임**
>
> URDF·TF 는 ROS 관례(REP-103)인 **FLU** 를 씀. 본 과목의 제어식은 Fossen 관례인 **FRD** 를 씀

축	ROS (URDF·TF, FLU)	본 과목 제어식 (FRD)	바꾸는 법
x	선수	선수	그대로
y	**좌현**	**우현**	$y_{FRD} = -y_{FLU}$
z	**위**	**아래**	$z_{FRD} = -z_{FLU}$

- 예: 좌현 추진기의 URDF 값은 $y_{FLU} = +1.027$ 이지만, 추력배분 행렬에는 $y_{b,L} = -1.027$ 로 들어감
- 위 장착 그림의 평면도는 **URDF 축(좌현 양수)** 으로 그려져 있음 — 5주차부터 쓰는 NED 식과 부호가 반대이므로 주의할 것
- 라이다 값을 쓸 때도 같음. 회피 코드 안의 점은 y 가 **좌현 양수**임 (1-4 절)

실측한 센서·추진기 배치

부품	x [m]	y [m]	z [m]	비고
3D 라이다 (이번 주차)	$+0.700$	0.000	$\mathbf{1.900}$	가장 높음. 360°
카메라 (좌·우 전방)	—	—	1.500	10주차 이후 사용
GPS	-0.850	0.000	1.300	
IMU	$+0.300$	-0.200	1.300	
추진기 좌·우	-2.374	± 1.027	0.318	5주차
추진기 선수	$+0.750$	0.000	0.318	6주차

- 위 값은 **VRX 실행 중 TF 에서 측정**한 것임. 추정치가 아님
- VRX 기본 라이다는 $z = 1.800$ · $y = -0.300$ 임 — 이번 주차는 둘 다 바꿈. 까닭이 아래 셋임

왜 $z = 1.90$ 인가 — 1.75 와 1.80 사이에 절벽이 있음

- 360° 라이다를 배 위에 달면 광선 일부가 **자기 배**를 때림. 그 값은 장애물이 아닌데 회피기는 장애물로 받음
- 그러면 **가만히 떠 있어도 회피 모드가 켜진 채로** 있게 됨
- 그래서 높이를 바꿔 가며 "들어온 값 중 선체 외곽 안쪽에서 온 비율" 을 셈 (`probe_lidar_pose.sh`)

```
높이z 자기배 수면 쓸수있음 가장가까운
1.65        82%      0%      25      35.30
1.70        82%      0%      25      35.32
1.75        81%      0%      25      35.33
```

높이 z [m]	자기 배 비율	판정
1.65	**82 %**	들어온 값 넷 중 셋이 자기 배임 — 쓸 수 없음
1.70	**82 %**	같음
1.75	**81 %**	같음
1.80 · 2.20 · 2.60	**0 %**	갑자기 사라짐

- 1.75 와 1.80 사이에서 82 % 가 0 % 로 **한 번에** 떨어짐. 서서히 줄지 않음 — 즉 **무엇인가 하나**가 가리고 있다는 뜻임
- 범인은 `cpu_cases_link` 임

항목	값
관절 원점 z	1.530 m
상자 크기	$0.59 \times 0.83 \times 0.47$ m
윗면 높이	$1.530 + 0.47/2 = \mathbf{1.765}$ m

- 라이다를 1.765 m 아래에 달면 수평 광선이 이 상자의 측면을 때림. 위로 올리면 상자 위를 지나감
- 그래서 **1.80 이면 되는데 왜 1.90 인가** — 파도에 기울기 때문임

라이다 높이	장비함 윗면과의 여유	그 여유가 사라지는 횡경사
1.80 m	3.5 cm	약 5°
1.90 m	13.5 cm	약 $\mathbf{18^\circ}$

- 5° 는 잔파도에서도 나옴. 18° 면 이 월드에서는 거의 안 나옴 → 1.90 으로 둠

> **수면 반사는 문제가 아니었음**
>
> - 흔히 "라이다를 낮추면 물에 맞아 헛점이 생긴다" 고 함. 이 월드에서는 **모든 높이에서 수면 비율이 0 %** 였음
> - 가장 아래 광선(-15°)이 수면과 만나는 거리는 $1.9 / \tan 15^\circ = 7.09$ m 임 — 즉 물리적으로는 고리가 생길 자리가 있음
> - 그런데 VRX 의 수면은 라이다에 **되돌아오지 않는 재질**임. 실제 바다와 다른 점이고, 실선 시험에서는 다시 확인해야 하는 항목임

높이를 올리면 잃는 것

항목	1.80 m	1.90 m
0.5 m 부표를 처음 보는 거리	4.85 m	5.2 m

- 높이면 **멀리** 보지만 **발밑**을 못 봄. 올릴수록 근거리 사각지대가 커짐
- 위 표는 그 대가가 35 cm 라는 뜻임. 호 다발이 보는 거리는 앞으로 4.5 m 뿐이므로 (1-5 절) 이 정도는 감수함

왜 $x = 0.70$, $y = 0$ 인가

좌표	값	까닭
x	$+0.70$	선수추진기($+0.75$) **바로 위**. 앞쪽에 둘수록 전방 사각지대가 작아짐. 더 앞으로 보내면 매달 구조물이 없음
y	0	VRX 기본값은 -0.3 임. 360° 센서를 중심선에서 30 cm 옮기면 **좌현과 우현이 서로 다른 거리로 보임** — 좌·우를 고르는 판단에 편향이 들어감

- 라이다 마스트 길이는 $z - 1.2965 - 0.05$ 로 계산되므로 z 는 1.35 m 보다 커야 함. 1.90 은 이 제약을 만족함

1-3. 라이다의 성능은 RTF 가 아니라 발행 주기가 정함

- 라이다를 달면 Gazebo 서버가 매 스캔마다 장면을 렌더함. 그래서 느려질 것이라고 예상함
- 그런데 **무엇이 느려지는지**를 가려야 함. 둘은 다른 문제임

재는 것	뜻	낮으면
RTF (real-time factor)	모의가 실제 시간을 얼마나 따라가는가. **1.0** = 같은 속도	제어 주기가 설계대로 돌지 않음
발행 주기 (points Hz)	라이다가 실제로 몇 번 자료를 내는가	회피기가 **낡은 장면**을 보고 판단함

라이다 세 판

판	빔 × 샘플	점/스캔	최대거리	설계 주기	쓰는 자리
`full`	**16 × 1875**	**30,000**	**130** m	**10** Hz	VRX 원본 그대로. 비교용
`lite`	**16 × 440**	**7,040**	**40** m	**10** Hz	**기본값**
`ultralite`	**8 × 220**	**1,760**	**30** m	**5** Hz	RTF 가 **0.3** 아래일 때

재 본 결과 (2026-10-27 · headless · 판마다 20초)

```
판          점/스캔   RTF_단독   RTF_구독중   points_Hz
none          0       0.99         -          -
ultralite     1760    0.99         0.99       4.97
lite          7040    0.99         0.99       9.96
full          30000   0.99         0.99       0.62
```

- 읽는 법 셋

관찰	뜻
RTF 가 네 판 모두 **0.99**	**RTF 는 제약이 아님**. 라이다를 달아도 모의 속도는 떨어지지 않음
`lite` 의 발행이 **9.96** Hz (설계 **10**)	설계대로 나옴 — 쓸 수 있음
`full` 의 발행이 **0.62** Hz	설계 **10** Hz 의 **1/16**. 한 장에 **1.61** s 걸림

> ★ `full` 판을 쓰면 회피기가 **멈춤 상태로 고정됨**
>
> - 회피기에는 `sys.sensor_timeout_sec` = **0.5** s 가 있음. 마지막 스캔이 **0.5** s 보다 오래되면 "센서가 끊겼다" 로 보고 **속력 0** 을 냄
> - `full` 의 스캔 간격은 **1.61** s 임 — 즉 **항상** 문턱을 넘음. 배가 아예 안 움직임
> - 그래서 기본값은 `lite` 임. 고른 근거는 추측이 아니라 위 표임
> - RTF 만 보고 "**0.99** 니까 `full` 도 되겠지" 라고 판단하면 틀림. **두 숫자를 같이 봐야 함**

> ℹ **RTF 를 왜 두 번 재는가**
>
> - 구독자가 붙으면 점구름을 **받아서 푸는** 비용이 따로 듦. `full` 은 한 장이 약 **960** KB 임
> - 그래서 "아무도 안 볼 때" 와 "한 명이 볼 때" 를 나눠 재어 둠. 실제 데모는 뒤쪽 숫자로 돎
> - 이 월드에서는 둘이 같았음(**0.99**) — 즉 받아 푸는 비용은 무시할 수 있었음

1-4. 라이다 값을 장애물 점으로 바꾸는 법

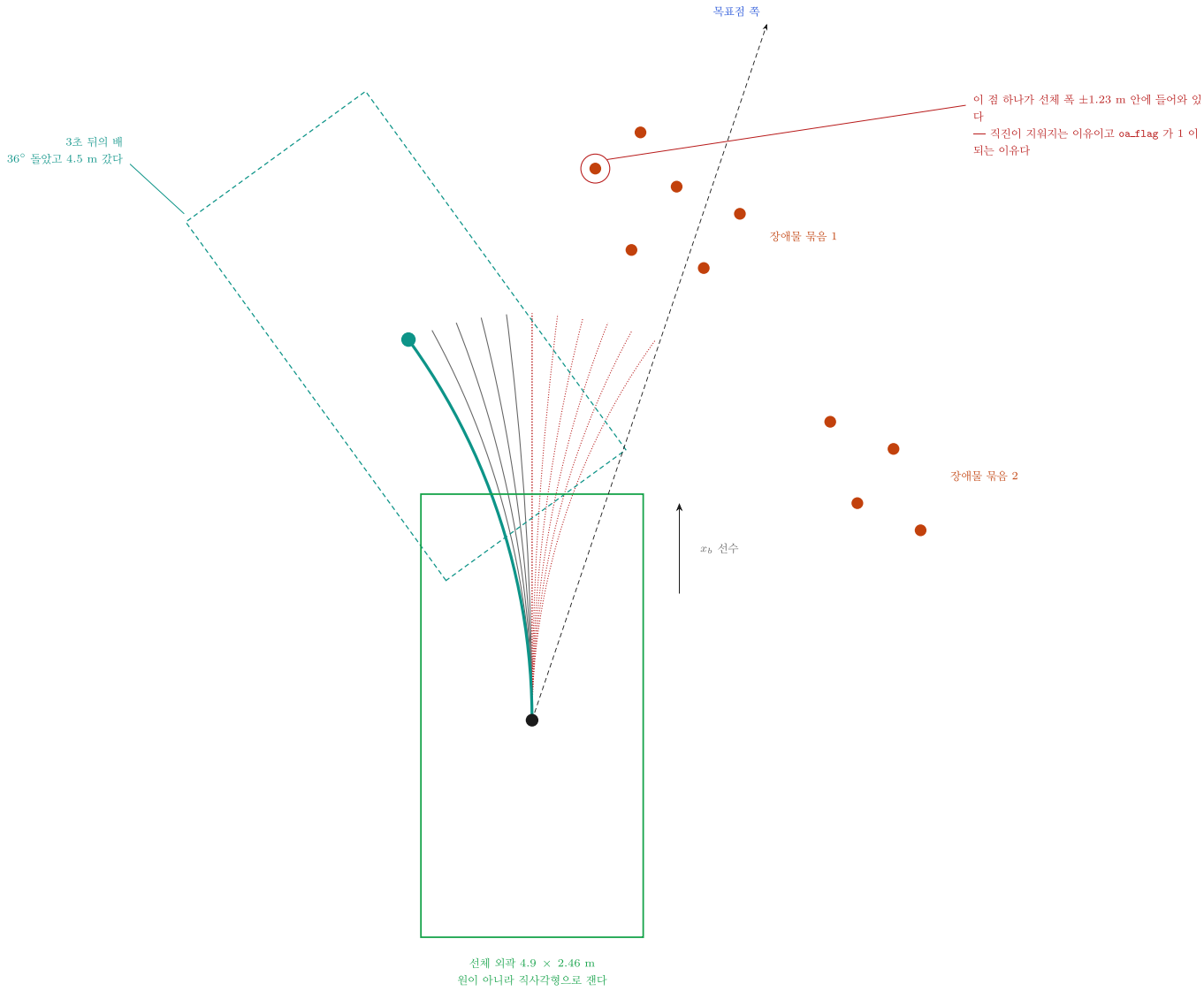
- 회피기는 거리값 배열을 바로 쓰지 않음. **선체 기준 2차원 점 묶음**으로 바꿈 (`scan_to_points`)
- 걸음 셋

걸음	하는 일	매개변수 (기본값)
① 걸러내기	`inf` · `nan` 을 버리고, 너무 가깝거나 너무 먼 값을 버림	`lidar.ignore_radius_m` $= 0.6$ · `lidar.max_range_m` $= 10.0$
② 좌표 바꾸기	각도·거리를 x(선수) · y(좌현) 로 바꿈	—
③ 군집	이웃한 값들을 묶고, **점이 적은 묶음은 버림**	`cluster_min_points` $= 3$ · `cluster_max_gap_m` $= 0.45$

- ① 의 `ignore_radius` $= 0.6$ m 는 **자기 배를 지우는 마지막 안전장치**임. 1-2 절에서 높이로 이미 해결했으므로 평소에는 할 일이 없음
- ① 의 `max_range` $= 10$ m 는 라이다 성능(40 m)보다 훨씬 짧음. 까닭은 1-5 절 끝에 있음
- ③ 이 중요함 — 혼자 떠 있는 점 하나는 **잡음**으로 보고 버림. 그렇게 하지 않으면 헛점 하나에 배가 피함

> **회피기가 쓰는 것은 `/scan` 한 고리뿐임**
>
> - 라이다는 토픽 둘을 냄 — `/points` (`PointCloud2` , 열여섯 고리 전부) 와 `/scan` (`LaserScan` , **한 고리**)
> - 회피기는 `/scan` 을 씀. 즉 이 회피기는 **평면 회피**임 — 높이를 보지 않음
> - 물 위를 떠다니는 부표·배를 피하는 데에는 충분함. 다리 아래나 경사면에서는 모자람
> - 열여섯 고리를 다 쓰면 무엇이 달라지는지는 다음 주차에서 다룸

1-5. 호 다발 21개 중 하나를 고름



붉은 점선 — 3초 안에 선체가 닿는 호. 점수를 매기지 않고 바로 지운다

회색 실선 — 살아남은 호. 이 중에서 점수로 고른다

굵은 청록 — 고른 호. 선회율 +12°/s, 반지름 7.16 m

점수 — 두 가지를 더한다

$$\text{score} = \underbrace{(1 - \text{align})}_{\text{목표를 향하는가}} + 0.3 \cdot \underbrace{\text{clip}\left(\frac{d_{min} - r}{3},\, 0,\, 1\right)}_{\text{여유가 넉넉한가}}$$

align 은 2초 뒤 헤딩이 목표 방위에서 얼마나 벗어나는가를 0~1 로 잰 것이고, d_{min} 은 **직사각형 선체에서 장애물 점까지의 거리**, $r = 0.60$ m 는 안전반지름, 3 m 는 "이만큼 여유가 있으면 만점" 의 기준이다. 가중치가 0.3 이므로 **목표 쪽으로 가는 것이 여유보다 세 배 무겁다** — 통로가 열려 있으면 굳이 넓은 쪽으로 돌지 않는다

3초는 4.5 m 다 — 자기 배 길이 4.9 m 보다 짧다. 호 다발 전체가 폭 2.7 m × 깊이 4.5 m 밖에 안 되는 손바닥만 한 부채다. 그래서 이 회피기는 **멀리 보는 기계가 아니라 가까이를 놓치지 않는 기계**다 — 라이다를 130 m 까지 늘려도 쓰이지 않는다 (`lidar.max_range_m` 기본값이 10 m 인 이유). 실제 제약은 선회율이 12°/s 뿐이라는 것이다. 3초에 36° 밖에 못 돈다

- 그림 읽는 법

요소	의미
가운데 초록 사각형	지금의 배. 외곽 4.9×2.46 m — **원이 아니라 직사각형**임
원점에서 뻗는 선 여러 개	깔아 둔 호. 선회율 $-12 \sim +12$ °/s 를 21개로 나눈 것 (그림에는 11개만)
붉은 점선	3초 안에 선체가 **닿는** 호. 점수를 매기지 않고 바로 지움
회색 실선	살아남은 호. 이 중에서 점수로 고름
굵은 청록	고른 호. 선회율 $+12^\circ$/s, 반지름 7.16 m
청록 점선 사각형	3초 뒤의 배. 36° 돌았고 4.5 m 갔음
붉은 점 묶음 둘	장애물 묶음. 군집을 거쳐 남은 점들
동그라미 친 점 하나	선체 폭 ± 1.23 m 안에 들어와 있는 점 — **직진이 지워지는 이유**이고 `oa_flag` 가 1 이 되는 이유
파란 점선 화살표	목표점 방향
아래쪽 글상자	점수식과 그 뜻

걸음 셋 — 깔고, 지우고, 점수를 매김

① **깔기** — 배가 **실제로 낼 수 있는** 선회율 21개를 미리 깔아 둠

- 자유롭게 "어디로 갈까" 를 고르지 않음. 낼 수 있는 선회율만 후보로 둠
- 선회율 r 로 속력 u 로 돌면 반지름 $R = u/r$, 시간 T 뒤 선회각 $\Phi = rT$

선회율 r [°/s]	반지름 R [m]	3초 뒤 선회각 Φ	호 길이
12.0	7.16	36.0°	4.5 m
9.6	8.95	28.8°	4.5 m
7.2	11.94	21.6°	4.5 m
4.8	17.90	14.4°	4.5 m
2.4	35.81	7.2°	4.5 m
0	∞ (직진)	0°	4.5 m

- $u = 1.5$ m/s · $T = 3$ s 이므로 호 길이는 모두 4.5 m 임. 호마다 15 걸음으로 나누어 따라감
- 함수 `simulate_arc_bank` 는 점 좌표만 내지 않고 **각 점에서의 선수각**도 함께 냄. 까닭은 바로 아래임

② **지우기** — 3초 안에 선체가 닿는 호를 지움

★ **선체를 원으로 재면 못 지나감**

- 배를 반지름 5 m 짜리 원으로 근사하면, 폭 10 m 아래의 통로는 **전부** 막힌 것으로 나옴
- 실제 WAM-V 는 길고 좁음 — 길이 4.9 m, 폭 2.46 m
- 그래서 `footprint_clearance` 는 **직사각형에서 점까지의 거리**를 잼. 직사각형은 방향이 있으므로 **각 점에서의 선수각**이 필요함 — ① 에서 선수각을 함께 낸 이유임
- 코드 상수는 WAM-V URDF 의 충돌 형상에서 가져온 것임

```
HULL_X_MIN, HULL_X_MAX = -2.4, 2.5
HULL_HALF_WIDTH = 1.23
```

- 원 근사도 코드에 남아 있음(`arc_collides` 의 `head is None` 경로) — 둘을 바꿔 가며 비교해 보라고 남긴 것임 (과제 ②)

③ **점수 매기기** — 살아남은 호 중 하나를 고름

$$\text{score} = \underbrace{(1 - \text{align})}_{\text{목표를 향하는가}} + 0.3 \cdot \underbrace{\text{clip}\left(\frac{d_{min} - r}{3}, 0, 1\right)}_{\text{여유가 넉넉한가}}$$

기호	뜻	값
align	2초 뒤 선수각이 목표 방위에서 얼마나 벗어나는가를 $0 \sim 1$ 로 잰 것	`plan.control_lookahead_sec` $= 2.0$
d_{min}	**직사각형 선체에서 장애물 점까지**의 최소 거리	—
r	안전반지름	`robot.safety_radius_m` $= 0.60$
3	"이만큼 여유가 있으면 만점" 의 기준	`plan.clearance_scale_m` $= 3.0$
0.3	여유에 주는 가중치	`plan.clearance_weight` $= 0.30$

- 가중치가 0.3 이므로 **목표 쪽으로 가는 것이 여유보다 세 배 무거움** — 통로가 열려 있으면 굳이 넓은 쪽으로 돌지 않음
- 점수가 같을 때는 선회율이 작은 쪽을 고름(`- 1e-6 * |rates|`) — 쓸데없이 돌지 않게 하는 미세 항임

왜 align 을 2초로 재는가

- 호는 3초 앞까지 따라가지만, 명령으로 내보내는 것은 2 초 뒤의 선수각임 (`CTRL_T`)
- 이 둘이 어긋나 있던 적이 있음 — 평가한 호, 점수를 매긴 시점, 내보낸 명령이 **서로 다른 시간축** 위에 있었음
- 지금은 셋을 하나로 맞춰 둠. 짧게 잡으면(예: 0.2 s) $12^\circ/\text{s} \times 0.2 = 2.4^\circ$ 밖에 못 돌아 조타가 사실상 안 됨

전부 막혔을 때 — 탈출

- 21개가 모두 지워질 수 있음. 그러면 속력을 $\max \times 0.30$ 으로 낮추고 (`robot.min_speed_ratio`) 호를 0.3 배로 줄여 다시 잼
- 고르는 기준이 바뀜 — **여유 + 0.6× 목표 정렬** (`plan.blocked_goal_weight` $= 0.6$)

⚠ **여유만 보고 고르면 배가 목표를 등지고 섬**

- 처음에는 "가장 넓은 쪽" 만 골랐음. 그러자 넓은 쪽이 **목표 반대 방향**인 자리에서 배가 그쪽을 보고 멈춰 있었음
- 실제로 380 초를 그렇게 보냈음. 막혀 있으니 호는 계속 지워지고, 가장 넓은 쪽은 계속 뒤쪽이었음
- 그래서 탈출에도 목표 정렬 항을 넣음. 같은 실수를 본인 알고리즘에서도 하기 쉬운 지점임

속력은 여유에 비례해서 줄임

선체 여유	속력
$\leq$ `safety_margin_m` (1.20 m)	max × 0.30
$\geq$ 1.20+ `slow_scale_m` (4.0 m)	max (1.5 m/s)
그 사이	1차로 비례

- 예전에는 1.5 와 0.45 두 값만 있었음 — 문턱을 넘을 때 속력이 **툭** 바뀌어 배가 끄덕였음
- 이력(hysteresis)도 있음 — 고른 호는 새 후보가 `plan.hysteresis_margin` = 0.10 만큼 더 좋지 않으면 **바꾸지 않음**. 없으면 비슷한 점수의 두 호 사이에서 조타가 떪

> ★ 3초는 4.5 m — **자기 배 길이 4.9 m 보다 짧음**
> - 호 다발 전체가 폭 2.7 m × 깊이 4.5 m 밖에 안 되는 손바닥만 한 부채임
> - 그래서 이 회피기는 **멀리 보는 기계가 아니라 가까이를 놓치지 않는 기계**임
> - 라이다를 130 m 까지 늘려도 쓰이지 않음 — `lidar.max_range_m` 기본값이 10 m 인 이유임
> - 실제 제약은 선회율이 12°/s 뿐이라는 것임. 3초에 36° 밖에 못 돎
> - 더 멀리 보고 싶으면 늘릴 것은 라이다가 아니라 `plan.sim_time_sec` 임 (과제 ②)

모드 둘 — GOAL 과 HOLD

모드	들어가는 때	통로가 열려 있으면	막혀 있으면
GOAL	목표점이 들어와 있음	목표 쪽으로 직진	호 다발에서 하나 고름
HOLD	목표점이 없음	제자리에 섬	장애물 **반대쪽**으로 0.6 m/s 로 비킴

- 이번 주차 데모는 GOAL 모드만 씀 — 목표점을 Simulink 가 계속 내기 때문임
- 목표점에 2 m 안으로 들어가면 속력 0 을 내고 `oa_flag` 를 내림

1-6. `oa_flag` 는 거리 문턱이 아님

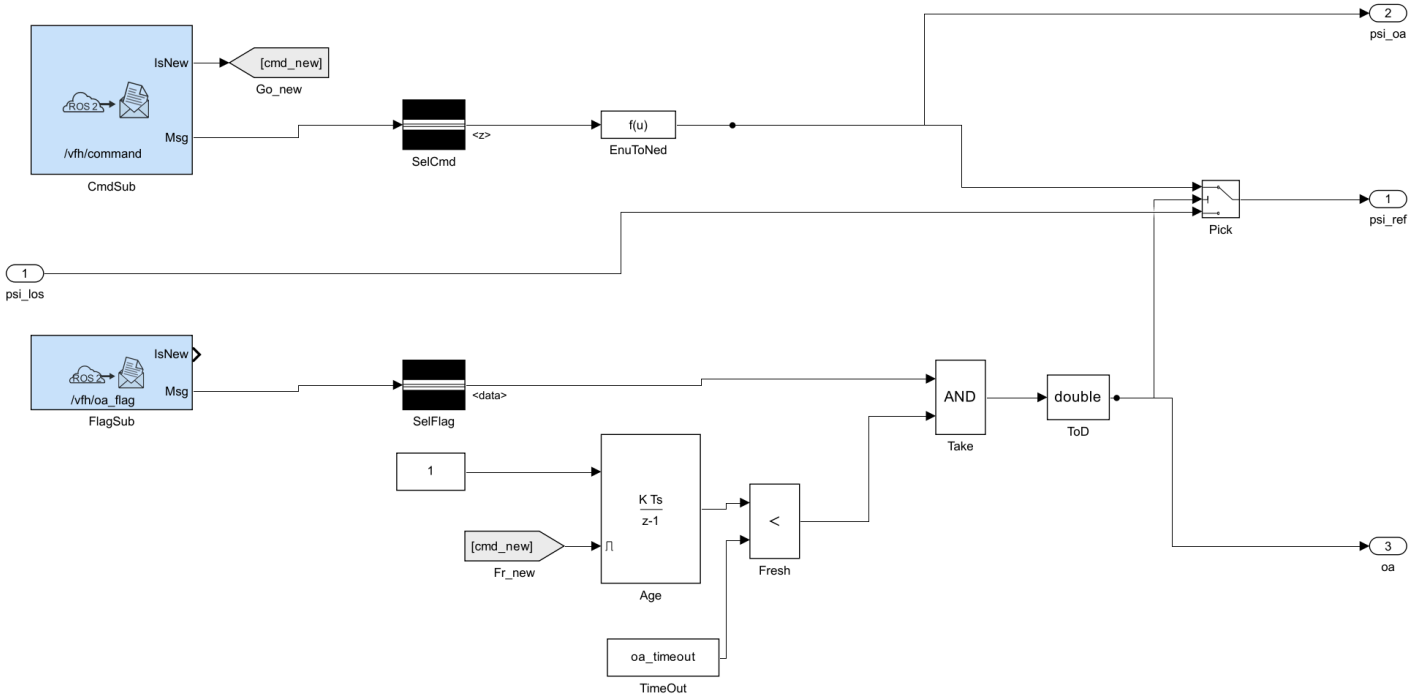
- 가장 많이 틀리는 지점임. `oa_flag` 는 "장애물이 몇 m 안에 있는가" 의 답이 **아님**
- **"목표까지의 직진 통로가 막혔는가"** 의 답임

```
dclear = 직진 경로를 따라갔을 때의 선체 최소 여유
dclear >= MARGIN (1.20 m)  →  oa_flag = false   (LOS 가 그대로 간다)
dclear <  MARGIN           →  oa_flag = true    (파이썬이 조타를 잡는다)
```

상황	가장 가까운 장애물	`oa_flag`	까닭
좌현 8 m 에 부표, 앞은 비었음	8 m	**0**	직진 통로가 열려 있음
정면 30 m 앞이 벽	30 m	**1**	직진하면 결국 닿음
선미 3 m 에 부표	3 m	**0**	가는 방향이 아님

- 즉 플래그가 0 인데 장애물이 가까이 있는 것은 **정상**임. 거리를 보지 않기 때문임
- 로그를 읽을 때도 이것을 기억해야 함 — "8.7 m 인데 회피가 꺼졌다" 는 버그가 아님

1-7. 이음새 — 스위치 하나와 명령의 나이 하나



이음새 — 제어기에 들어가는 명령을 누가 내는가.

Pick 의 2번(판단)이 0.5 보다 작으면 3번(LOS) 을, 크면 1번(회피) 을 낸다.
판단 = oa_flag AND (명령이 끊긴 지 oa_timeout 초보다 적다)

토픽 둘 — 둘 다 받기만 한다. 이 모델은 회피 쪽으로 아무것도 보내지 않는다.
/vfh/command geometry_msgs/Twist angular.z = 절대 방위각 (ENU, rad)
/vfh/oa_flag std_msgs/Bool 피하는 중이면 true

oa_flag 는 거리 문턱이 아니다. "목표까지 직진 통로가 막혔는가" 의 답이다.
그래서 장애물이 8 m 옆에 있어도 통로가 열려 있으면 0 이고,
30 m 앞이 막혀 있으면 1 이다.

- 그림 읽는 법

요소	의미
왼쪽 위 `CmdSub`	`/vfh/command` 를 받음. 출력 둘 — **1번** `IsNew`, 2번 `Msg`
`SelCmd` → `EnuToNed`	`angular.z` 를 뽑아 ENU 방위각을 NED 선수각으로 돌림
왼쪽 아래 `FlagSub`	`/vfh/oa_flag` 를 받음
가운데 `Age` (적분기)	마지막 명령이 **들어온 지 몇 초 지났는가**
`Fresh` (<) · `Take` (AND)	판단 — 플래그가 1 **이고** 명령이 신선한가
오른쪽 `Pick` (스위치)	위 입력 = 회피, 가운데 = 판단, 아래 = LOS

- 하는 일은 한 줄임 — **제어기에 들어가는 명령을 누가 낼지 고름**

```
판단 = oa_flag  AND  (명령이 끊긴 지 oa_timeout 초보다 적다)

판단 = 0  →  psi_ref = psi_los   (5주차와 완전히 같음)
판단 = 1  →  psi_ref = psi_oa    (파이썬이 낸 값)
```

왜 "나이" 를 재는가

- 파이썬 노드가 죽거나 WSL 이 버벅대면 `/vfh/command` 가 끊김. 그런데 `oa_flag` 의 마지막 값은 `true` 로 남아 있음
- 그 상태로 스위치를 두면 **낡은 회피 명령을 계속 쓰는** 배가 됨 — 가장 위험한 고장임

- 그래서 명령이 들어올 때마다 적분기를 0 으로 되돌리고(`ExternalReset`), 그 값이 `oa_timeout` = **0.5** s 를 넘으면 **LOS 로 되돌아감**
- 즉 이음새의 기본 상태는 **5주차 동작**임. 회피는 "증거가 있을 때만" 끼어듦

> ⚠ **적분기 되돌림은 `'level'` 이지 `'rising'` 이 아님**
>
> - `IsNew` 는 새 메시지가 있는 **스텝 동안 1** 로 유지되는 신호임. 상승에지만 보면 놓치는 스텝이 생김
> - Discrete-Time Integrator 의 외부 되돌림 포트는 블록 **왼쪽** 변에 붙음 — 배선할 때 헷갈리는 지점임

ENU 와 NED 를 바꾸는 한 줄

- 파이썬은 ENU(동쪽 0, 반시계 양수)로 냄. 본 과목 제어기는 NED(북쪽 0, 시계 양수)로 돎

$$\psi_{NED} = \mathrm{ssa}\left(\frac{\pi}{2} - \psi_{ENU}\right)$$

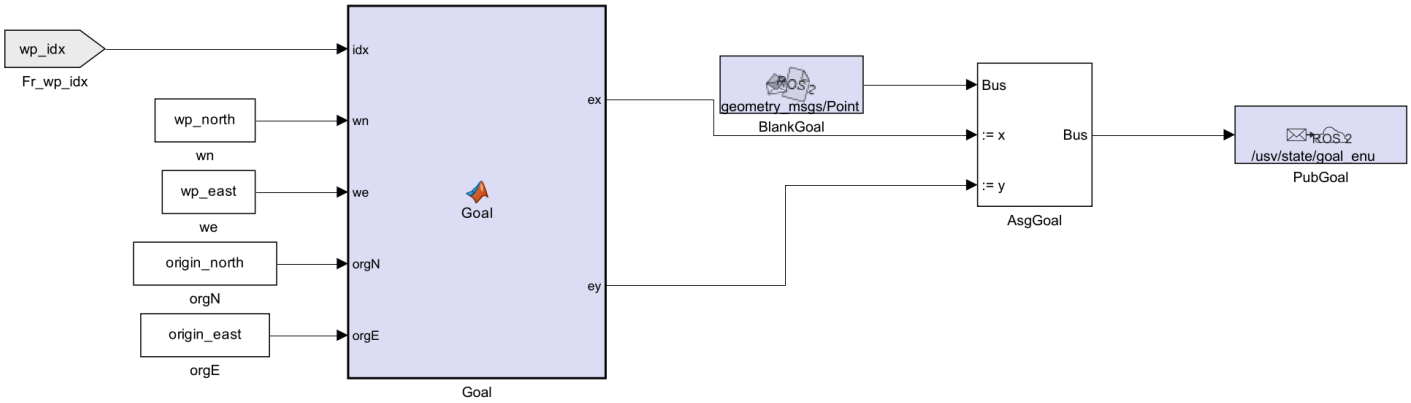
```
atan2(sin(pi/2-u), cos(pi/2-u))
```

- `atan2` 로 감싸는 것은 결과를 $-\pi \sim \pi$ 로 **접기** 위함임 (5주차 `ssa` 와 같은 일)
- 이 변환은 **자기 역함수**임 — 두 번 걸면 원래로 돌아옴. 그래서 부호를 틀렸는지 확인하기 쉬움
- 이 한 줄은 MATLAB Function 안에 숨기지 않고 `Fcn` 블록으로 **블록도에 보이게** 둠

왜 모델을 새로 만들지 않고 복사했는가

- `build_w09_models.m` 은 `W05_1_vrx.slx` 를 **복사**해서 시작함. 새로 그리지 않음
- 다시 그리면 "제어기가 5주차와 같다" 는 것을 **주장**할 수 있을 뿐임. 복사하면 **같은 블록**임
- 복사한 뒤 하는 일 셋뿐

하는 일	내용
끊기	`Guidance → InnerLoop` 가지 하나를 끊음
끼우기	그 자리에 `AvoidSeam` 을 넣음. 기존 태그 `psi_ref` 는 `psi_los` 로 이름만 바꿈
더하기	`GoalPublisher` 를 붙임 (아래)



목표점 발행 — 파이썬 회피기가 "앞이 막혔는가" 를 판정하는 데 쓴다.
/usv/state/goal_enu geometry_msgs/Point x = 동, y = 북 (월드 ENU)

좌표계가 여기서 한 번 바뀐다.
시뮬링크 스폰 원점 기준 NED — wp_north(k), wp_east(k)
파이썬 월드 ENU — x = 동 + origin_east, y = 북 + origin_north

같은 일을 하는 파이썬 노드(goal_publisher_enu) 는 띄우지 않는다.
둘이 같은 토픽에 쓰면 서로를 덮어쓴다.

요소	의미
`Fr_wp_idx`	지금 몇 번째 웨이포인트를 보고 있는지 (5주차 유도단이 내는 값)
`Goal`	그 웨이포인트를 **월드 ENU 좌표**로 바꿈
`Publish`	`/usv/state/goal_enu` 로 1 Hz 로 냄

- 좌표를 바꾸는 식 둘 — 경로 좌표는 원점이 출발점이지만 ENU 는 원점이 월드임

$$x_{ENU} = y_{wp} + \text{origin_east}, \qquad y_{ENU} = x_{wp} + \text{origin_north}$$

- 즉 목표점을 내는 쪽도 **Simulink** 임. 파이썬은 받기만 함 — 이 주차의 설계에서 유도의 주인은 끝까지 Simulink 쪽에 있음

1-8. 자를 하나로 씀 — `ca_bridge`

- 회피기는 `/wamv/gps_pose` 를 봄. Simulink 는 `/wamv/sensors/position/ground_truth_odometry` 를 봄
- 연구실 원본은 `/wamv/gps_pose` 를 **GPS 위경도를 평면으로 펴서** 만들었음. 그런데 그 평면의 원점은 **처음 받은 위치**임
- Simulink 가 쓰는 참값 오도메트리의 원점은 **Gazebo 월드 원점**임
- 자가 둘이면 같은 고리 안에서 **회피의 "왼쪽" 과 Simulink 의 "왼쪽" 이 다름** — 몇 m 씩 어긋남

쪽	원래 쓰던 자	원점
파이썬 회피기	GPS 위경도를 편 평면	처음 받은 위치
Simulink	참값 오도메트리	Gazebo 월드 원점

- 그래서 `ca_bridge.py` 를 끼움. 하는 일은 **참값 오도메트리를** `PoseStamped` **로 모양만 바꿔** `/wamv/gps_pose` 로 다시 내는 것뿐임
- 회전도 변환도 없음 — 둘 다 ENU 이기 때문임. 코드가 짧은 것이 이 노드의 요점임
- 결과: 양쪽이 **같은 토픽의 같은 숫자**를 봄

> **ℹ 실선에서는 이 브리지를 쓸 수 없음**
> - 참값 오도메트리는 시뮬레이터만 주는 값임. 실제 배에는 없음
> - 실선에서는 양쪽이 **같은 항법 출력**(예: GPS/INS 융합 결과)을 쓰도록 맞춰야 함
> - 이 주차에서 브리지로 해결한 문제는 사라지지 않고, **항법단을 하나로 두는 일**로 옮겨 감

1-9. 그림은 누가 그리는가 — 파이썬

- 극좌표 실시간 그림은 **파이썬**(`ca_viz.py`)이 그림. Simulink 가 그리지 않음. 까닭 셋

까닭	설명
자료가 이미 그쪽에 있음	스캔 440 개를 Simulink 로 가져오려면 매 스텝 **가변 크기 신호**로 풀어야 함. 제어와 무관한 비용임
Scope 는 시간축 그림임	극좌표 산점도를 그리려면 MATLAB Function 블록이 필요함 — 모델이 그림을 그리는 일을 하게 됨
모델은 **자기만 아는 값 둘**만 내면 됨	`/sim/psi_los` 와 `/sim/psi_cmd` — Publish 블록 둘로 끝남

- 즉 **그리는 일은 파이썬, 그릴 값을 내는 일은 양쪽**임
- 보고서용 그림은 여전히 MATLAB(`W09_plot.m`)이 그림 — 실행이 끝난 뒤 로그로 그리는 것은 Simulink 쪽이 유리함

극좌표에 겹쳐 그리는 선 넷

선	담는 것	어디서
파랑 실선	지금 선수각	오도메트리. 배 기준이므로 **늘 위쪽 0°** 에 고정됨
초록 점선	LOS 헤딩 명령	`/sim/psi_los` (5주차 유도)
자홍 점선	회피 헤딩 명령	`/vfh/command` (이번 주차)
검정 얇은 선	Simulink 가 **실제로 쓰는** 값	`/sim/psi_cmd` — 스위치를 통과한 뒤의 값

- 읽는 법이 이 그림의 전부임

보이는 것	뜻
검정이 **초록** 위에 겹침	평상시 — LOS 가 조타함
검정이 **자홍** 위에 겹침	회피 중 — 파이썬이 조타함
검정이 **어느 쪽에도** 안 겹침	**이음새가 틀렸음** — 이것을 보려고 넷을 겹쳐 그림

2부 · 실습

무엇을 돌리는가 — 폴더 둘

> ✎ **치는 것은 터미널 둘과 MATLAB 한 줄임**
> 나머지 스크립트는 **확인용**임. 처음 돌릴 때는 A → C → D → G 만 해도 데모가 됨

`W09_vrx_lite/` — WSL 쪽

파일	하는 일	언제
`make_wamv_ca.sh`	라이다를 단 WAM-V URDF 를 만듦 (세 판)	실습 A
`run_vrx_ca.sh`	그 배로 VRX 를 띄움	실습 A · **터미널 1**
`probe_lidar.sh`	라이다가 무엇을 보는지 한 장 찍어 봄	실습 B
`probe_lidar_pose.sh`	높이를 훑어 자기 배 비율을 잼	실습 B
`measure_rtf.sh`	판마다 RTF 와 발행 주기를 잼	실습 C
`w09_ca.rviz`	Rviz2 설정 (Fixed Frame 이 박혀 있음)	실습 D
`check_obstacles.sh`	배를 밀면서 **가장 가까운 거리가 줄어드는지** 잼	실습 D
`ca_bridge.py`	참값 오도메트리를 `/wamv/gps_pose` 로 다시 냄	실습 F · G
`ca_viz.py`	극좌표 실시간 그림	실습 H
`run_ca_nodes.sh`	위 셋을 한꺼번에 띄움	실습 F · G · **터미널 2**
`wamv_ca_components.xacro`	센서 배치 정의 (직접 치지 않음)	—

`W09_simulink/` — MATLAB 쪽

파일	하는 일	직접 치는가
`W09_setup.m`	이득·웨이포인트·장애물 표·원점을 작업공간에 올림	**예**
`W09_vrx_run.m`	토픽 점검 → RTF 측정 → `sim` → 그림 → 판정	**예**
`W09_1_vrx.slx`	모델. 유도 둘 + 이음새 + 5주차 제어기	열어서 봄
`W09_plot.m`	결과 그림	아님 (`W09_vrx_run` 이 부름)
`build_w09_models.m`	모델을 **다시 만듦**	아님 (실습 I)

> ⚠ **아래 "정상 출력" 의 숫자는 노트북 성능과 월드에 따라 달라짐**
> - 표로 실은 측정값은 2026-10-27 에 실제로 찍은 것임
> - 본인 숫자가 다르더라도 **모양이 같으면 정상**임. 과제에는 본인 숫자를 적을 것

A. 실습 1 — 가제보 모델이 무엇이고 어떻게 띄우는가 (20분)

A-1. 배가 무엇으로 조립되는가

- WAM-V 는 하나의 파일이 아님. **xacro 매크로**를 쌓아 URDF 를 만들어 냄

층	파일	담는 것
뼈대	`wamv_gazebo.urdf.xacro`	선체·부력·물리. VRX 가 주는 것
추진기	`../W06_vrx_lite/wamv_bow_thrusters.xacro`	추진기 셋의 자리 — **6주차 것을 그대로 씀**
센서	`wamv_ca_components.xacro`	GPS · IMU · 참값 · **3D 라이다**

- 즉 이번 주차에 새로 쓴 것은 **센서 층 하나**임. 배를 다시 만들지 않음
- `make_wamv_ca.sh` 가 6주차 `make_wamv_bow.sh` 와 다른 곳은 **두 줄**임 — 센서 층을 끼우라는 xacro 인수 둘

A-2. URDF 를 만듦

```
cd ~/Capstone-Design/1_2026-2학기_강의자료/10-주차별-강의자료/W09_vrx_lite
bash make_wamv_ca.sh
```

- 정상 출력 (끝 두 줄)

```
라이다 자리 : x = 0.700  y = 0.000  z = 1.900   (post 0.63 + arm 0.03 + sensor 0.04)
만들었습니다 : /home/<사용자명>/capstone_ws/wamv/wamv_ca_lite.urdf
```

> ⚠ **라이다 x 는 한 숫자가 아니라 세 관절의 합임**
> - URDF 안에서 라이다는 기둥(post) → 팔(arm) → 센서 세 관절로 매달려 있음
> - 그래서 `grep` 으로 `x=0.7` 을 찾으면 **안 나옴**. 스크립트가 세 관절 원점을 더해서 찍어 주는 이유임
> - 센서 자리를 바꿀 때는 이 합을 보고 확인할 것

A-3. 띄움 — 터미널 1

```
bash ../W03_vrx_lite/stop_vrx.sh
bash run_vrx_ca.sh
```

- 정상 출력 (가운데쯤)

```
추진기 토픽 : /wamv/thrusters/left/thrust /wamv/thrusters/right/thrust /wamv/thrusters/bow/thrust
라이다 토픽 : /wamv/sensors/lidars/lidar_wamv_sensor/points
              /wamv/sensors/lidars/lidar_wamv_sensor/scan
창이 뜨면 /vrx/release 를 보내십시오 (아래 명령)
```

- Gazebo 창이 뜨고 배가 떠 있으면 성공임

> ⚠ **`/vrx/release` 를 안 보내면 배가 못 움직임**
> - VRX 는 경기 시작 전까지 배를 고정해 둠. 추력을 줘도 꼼짝 안 함
> - 터미널 2 에서 한 번 보냄. `run_ca_nodes.sh` 는 이것을 **대신 보내 줌**
>
> ```
> gz topic -t /vrx/release -m gz.msgs.Empty -p ''
> ```

> **⚠ 6주차와 다른 한 줄 — 렌더 엔진**
> - 6주차 URDF 에는 그림을 그리는 센서가 없어서 Gazebo **서버**는 아무것도 렌더하지 않았음
> - 라이다를 달면 서버가 매 스캔마다 장면을 그림. 그래서 `run_vrx_ca.sh` 는 **서버 엔진도** `ogre` 로 낮춤
> - 끄는 것이 아니라 낮추는 것임 — `ogre2` 는 WSL2 의 소프트 렌더에서 특히 비쌈
> - 이 한 줄이 없으면 창이 뜨다 멈추거나 RTF 가 0.2 아래로 떨어짐

A-4. 토픽이 다 올라왔는지

```
ros2 topic list | grep -E "thrusters|lidar|ground_truth"
```

- 정상 출력 — 추진기 셋(각 `thrust`), 라이다 둘(`points` · `scan`), 참값 하나가 보이면 됨

자주 걸리는 지점	조치
아무것도 안 보임	`ROS_DOMAIN_ID` 가 비어 있음 — `export ROS_DOMAIN_ID=7` 뒤 다시
추진기는 보이는데 라이다가 없음	6주차 URDF 로 띄웠음 — `run_vrx_bow.sh` 가 아니라 `run_vrx_ca.sh` 임
`bow/thrust` 가 없음	선수추진기 층이 안 붙었음 — `make_wamv_ca.sh` 를 다시

B. 실습 2 — 라이다가 실제로 무엇을 보는가 (20분)

B-1. 한 장을 들여다봄

```
bash probe_lidar.sh
```

- 스스로 띄우고 찍고 끔. 보여 주는 것 넷

순서	보여 주는 것	무엇을 읽는가
①	라이다가 내는 토픽과 각 메시지형	`scan` 과 `points` 는 **다른 토픽**임
②	`/scan` 머리글 — 광선 수와 각도 범위	몇 개를 어느 범위로 쏘는가
③	거리값 분포 — 유효 개수·최소·중앙·최대·최다 구간	**값이 한 거리에 몰려 있는지**가 핵심임
④	`PointCloud2` 한 장의 크기와 `ros2 topic hz`	Rviz 에서 볼 것과 같은 자료임

> **★ ③ 에서 값이 한 거리에 몰려 있으면 그것은 장애물이 아니라 평면임**
> 둘 중 하나를 의심함
>
> | 몰린 자리 | 범인 | 확인 |
> |---|---|---|
> | 7 m 근처 | **수면** — $z = 1.9$ 라이다의 가장 아래 광선($-15°$)이 $1.9 / \tan 15° = 7.09$ m 앞에서 물과 만남 | 배를 돌려도 고리가 따라 옴 |
> | $0.35 \sim 2.3$ m | **자기 배** — 라이다가 너무 낮음 | 배를 돌려도 같은 모양이 그대로 있음 |
>
> - 이 월드에서는 $z = 1.9$ 에서 둘 다 나오지 않음. 나오면 높이를 의심할 것

B-2. 높이를 직접 훑어 봄

```
bash probe_lidar_pose.sh 1.65 1.70 1.75 1.80
```

- 한 높이에 2~3분 걸림. 화면 없이 돎
- 정상 출력 (2026-10-27 실측, 아래 셋)

```
높이z 자기배 수면 쓸수있음 가장가까운
1.65        82%      0%       25      35.30
1.70        82%      0%       25      35.32
1.75        81%      0%       25      35.33
```

열	뜻
자기배 %	선체 외곽(앞 **2.5** · 뒤 **−2.4** · 반폭 **1.23** m) 안에서 들어온 광선의 비율
수면 %	**5** m 넘는 거리에 한 구간으로 몰린 값의 비율
쓸수있음	자기 배도 수면도 아닌 값의 개수 — 진짜 장애물을 볼 여력
가장가까운	자기 배를 뺀 최소 거리

- **1.80** 부터 자기배가 **0 %** 로 떨어짐. 그 절벽의 범인이 `cpu_cases_link` 임 (1-2 절)
- 채택값은 **1.90** 임 — **1.80** 도 0 % 이지만 횡경사 여유가 **5°** 뿐임

> ⚠ **이 스크립트는 오래 걸림. 수업 중에는 표만 읽고 넘어갈 것**
> - 네 높이를 다 돌리면 10분이 넘음. 과제로 돌려 보는 것을 권함
> - 중간에 멈추려면 `Ctrl+C` 뒤 `bash ../W03_vrx_lite/stop_vrx.sh`

C. 실습 3 — RTF 와 발행 주기를 재서 판을 고름 (15분)

```
bash measure_rtf.sh
```

- 네 판을 차례로 잼 — `none` · `ultralite` · `lite` · `full`. 판마다 20초, 화면 없이 돎
- 정상 출력 (2026-10-27 실측)

```
판            점/스캔   RTF_단독   RTF_구독중   points_Hz
none             0       0.99          -          -
ultralite     1760       0.99       0.99       4.97
lite          7040       0.99       0.99       9.96
full         30000       0.99       0.99       0.62

RTF 1.00 = 실제 시간과 같은 속도.  points_Hz 설계값은 URDF 의 update_rate.
```

- 결론 셋

결론	근거
RTF 는 **제약이 아님**	네 판 모두 **0.99**. 라이다를 달아도 모의 속도가 안 떨어짐
실제 제약은 **발행 주기**임	`full` 이 설계 **10** Hz 의 **1/16** 인 **0.62** Hz 로 주저앉음
기본값은 `lite`	설계 **10** Hz 를 **9.96** Hz 로 지킴

> ⚠ `full` 판으로는 배가 **아예 안 움직임**
>
> - 회피기의 `sensor_timeout` 은 0.5 s 임. `full` 의 스캔 간격은 1.61 s 임
> - 즉 항상 "센서 끊김" 으로 판정되어 속력 0 을 냄. 오류 메시지 없이 가만히 있음
> - **RTF 만 보고 판을 고르면 이 함정에 빠짐.** 두 숫자를 같이 봐야 함

- 한 판만 재려면 `bash measure_rtf.sh lite`, 재는 시간을 바꾸려면 `SEC=30 bash measure_rtf.sh`

D. 실습 4 — Rviz2 에서 장애물이 `PointCloud2` 로 들어오는지 (25분)

D-1. 눈으로 확인

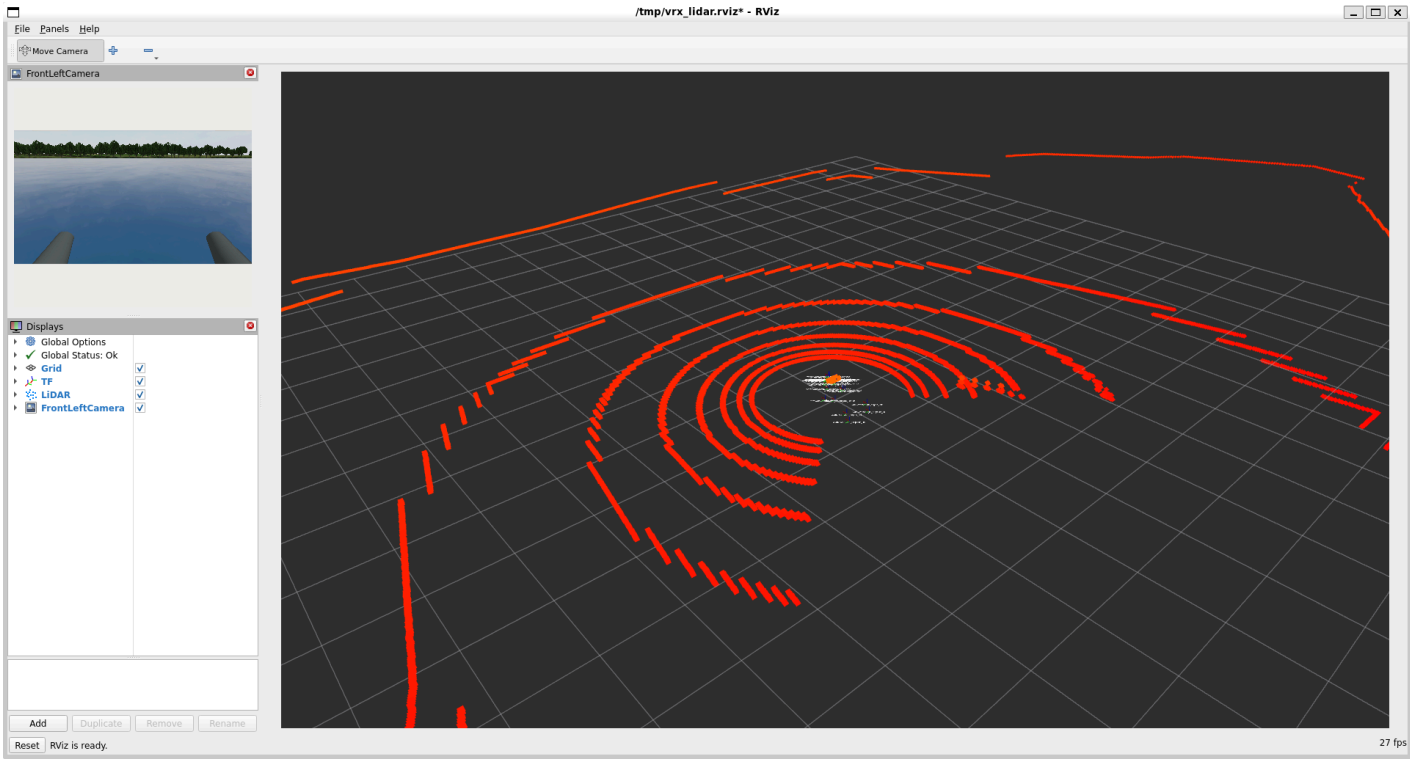
- 터미널 1 에 Gazebo 가 떠 있는 상태에서, 터미널 3 을 하나 더 엶

```
cd ~/Capstone-Design/1_2026-2학기_강의자료/10-주차별-강의자료/W09_vrx_lite
rviz2 -d w09_ca.rviz
```

> ★ Fixed Frame 은 반드시 `wamv/wamv/base_link` 임
>
> - Rviz2 기본값 `map` 은 **이 세계에 없는 좌표계**임. 그냥 두면 모든 표시가 `Status: Error` 로 뜸
> - `w09_ca.rviz` 안에 박아 두었으므로 `-d` 로 열면 손댈 것이 없음
> - 설정 없이 맨손으로 열었다면: Global Options → Fixed Frame → `wamv/wamv/base_link`

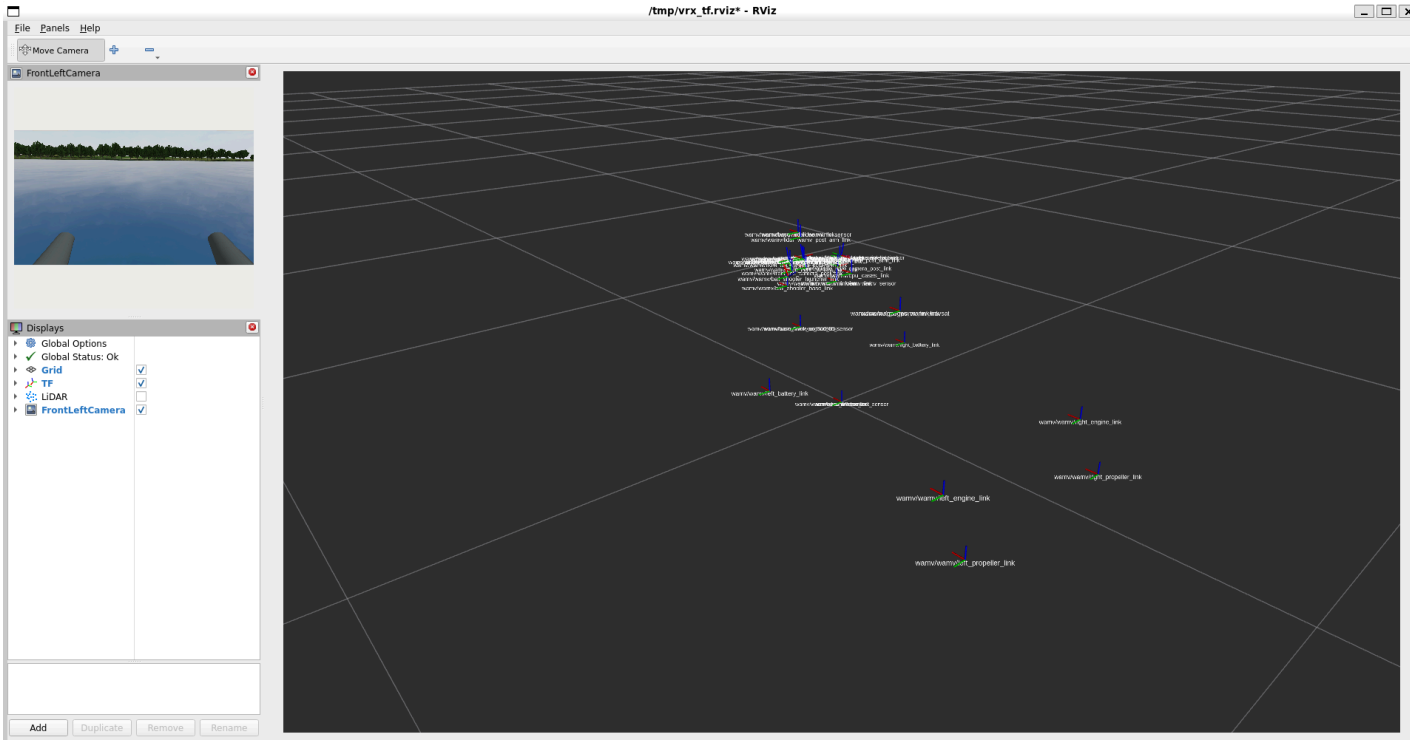
- 설정 파일이 켜 주는 표시 넷

표시	토픽	어떻게 보이는가
`Grid`	—	5 m 격자. 거리를 눈으로 재는 자
`PointCloud2 (장애물)`	`.../points`	높이에 따라 색이 변하는 점들 (`AxisColor` · Z)
`LaserScan (/scan, 회피가 쓰는 것)`	`.../scan`	붉은 구슬. **회피기가 실제로 보는 것**이 이것임
`TF (좌표계 나무)`	—	기본 꺼짐. 필요할 때 켬



- 그림 읽는 법

요소	의미
가운데 격자	5 m 간격. 배는 원점에 있음
색이 변하는 점 띠	`PointCloud2` . 색은 높이(Z)
멀리 보이는 긴 선	해안선 — 장애물이 아님
왼쪽 위 작은 화면	전방 카메라. 선체 두 개가 보임



- 그림 읽는 법

요소	의미
빨강 · 초록 · 파랑 축	x · y · z (ROS FLU 축이므로 초록이 **좌현**)
가운데 뭉친 축들	선체·라이다·GPS·IMU 가 모두 `base_link` 근처에 있음
뒤쪽에 떨어진 넷	좌·우 추진기와 그 프로펠러

- 배를 돌리거나 밀면 점들이 함께 돌아야 함. 돌지 않으면 Fixed Frame 이 월드에 고정된 것임

D-2. 화면이 비어 있을 때

증상	원인	조치
`PointCloud2` 가 비어 있음	토픽 이름 또는 Fixed Frame	토픽을 다시 고르고, Fixed Frame 을 `wamv/wamv/base_link` 로
`Fixed Frame [map] does not exist`	기본값을 그대로 씀	위와 같음
한 장 뜨고 멈춤	시뮬레이터가 멈춰 있음	Gazebo 창의 재생 버튼 · `/vrx/release`
카메라 화면이 회색	토픽 이름 오타	By topic 으로 다시 고름
느리지만 맞게 나옴	RTF 가 낮음	**오류가 아님** — 노트북 성능 문제임

ℹ VRX 센서 토픽의 QoS 는 모두 `RELIABLE` 임

- 실무에서는 "고빈도 센서는 `BEST_EFFORT` " 가 관례임. **이 시뮬레이터는 다름**
- `ros_gz_bridge` 가 기본 QoS 로 발행하기 때문임 — 발행자가 있는 토픽 25개가 전부 `RELIABLE` 임
- 확인하는 법

```
ros2 topic info /wamv/sensors/lidars/lidar_wamv_sensor/scan --verbose | grep -E "Publisher count|Reliability"
```

- 구독자를 `BEST_EFFORT` 로 만들면 **섞여도 받아짐**(호환 방향이 그쪽임). 회피기와 `ca_viz` 가 그렇게 되어 있음
- 반대로 발행이 `BEST_EFFORT` 인데 구독이 `RELIABLE` 이면 **한 장도 안 들어옴** — 이쪽이 자주 걸리는 함정임

D-3. 숫자로 확인 — 다가갈수록 줄어드는가

- 눈으로 보는 것만으로는 부족함. **가만히 서 있으면** 주위가 비어 있어 "값이 25개 들어왔다" 정도만 보임
- 회피가 되려면 **다가갈수록 거리가 줄어드는 것**이 보여야 함

```
bash check_obstacles.sh
```

- 배를 앞으로 밀면서 1초마다 가장 가까운 값을 적음
- 읽는 법 — `가장가까운` 이 40 m 에서 10 m 로 **단조롭게 줄면 성공**임

줄지 않을 때	원인
값이 처음부터 끝까지 같음	배가 안 움직임 — `/vrx/release` 가 빠졌음
값이 들쭉날쭉함	앞이 비어 있음 — 월드에서 장애물이 있는 쪽으로 돌릴 것
처음부터 2 m 아래로 고정	라이다가 **자기 배만** 봄 — 높이를 확인할 것 (1-2 절)

E. 실습 5 — 월드의 장애물 표를 직접 만듦 (10분)

- `W09_plot.m` 은 **월드에 실제로 있는 장애물 위치**를 궤적 위에 겹쳐 그림. 그 표를 만드는 법임
- 월드 파일에서 `<include>` 를 읽음

```
grep -c "<include>" /home/<사용자명>/vrx_ws/src/install/share/vrx_gz/worlds/stationkeeping_task.sdf
```

> ⚠ **주석 안의 `<include>` 를 먼저 지워야 함 — 2026-10-27 에 여기서 틀렸음**
>
> - 그냥 세면 **51개**가 나옴. 그 가운데 다섯은 `<!-- <include> ... -->` 안에 든 **주석**임 — 월드에 없는 물체임
> - 주석을 지우지 않고 세었더니 정북 $2 \sim 4$ m 앞에 부표가 둘 더 있는 것으로 읽혀 "두 번 막힌다" 는 결론이 흔들렸음
> - 주석을 먼저 지우면 **46개**, 경로 주변은 **42개**임
> - $x_n = 105 \cdot 117$ 줄은 주석 안에 들어 있어 **월드에 없음**

- 주석을 지우고 세는 법

```
python3 - <<'PY'
import re, pathlib
p = pathlib.Path.home() / 'vrx_ws/src/install/share/vrx_gz/worlds/stationkeeping_task.sdf'
s = re.sub(r'<!--.*?-->', '', p.read_text(), flags=re.S)
print('주석 뺀 include :', s.count('<include>'))
PY
```

- 정상 출력

```
주석 뺀 include : 46
```

- 나온 좌표를 경로 원점(`path_north0` $= 200$, `path_east0` $= -532$) 기준으로 옮기면 `W09_setup.m` 의 `obs_x_n` · `obs_y_n` 이 됨

> ⚠ **장애물 배치는 월드마다 다름**
>
> - 월드를 바꾸면 이 표를 다시 만들어야 함
> - `W09_setup.m` 의 표는 `stationkeeping_task` 전용임

이 월드에서 배가 가는 길

항목	값
웨이포인트 (북)	$0 \to 60 \to 125$ m
웨이포인트 (동)	$0 \to 0 \to 0$ — **정북 직선**
경로 원점 (월드)	북 200 · 동 -532
막히는 자리	$x_n = 33$ 의 벽 · $x_n = 93$ 의 슬라럼 한 줄

- 즉 **정확히 두 번 막힘**. 막힌 두 번만 파이썬이 조타를 잡고, 나머지는 Simulink 의 LOS 가 잡음
- 한 번 주행에 전환이 **네 번** 일어나므로 `oa` 신호를 그래프로 읽기에 좋음
- $x_n = 33$ 의 벽은 가운데가 열려 있음 — $y_n = -27 \sim +1$, 폭 28 m 짜리 문임
- 배는 그 문의 **오른쪽 끝에서 2 m** 떨어진 자리에서 뜸. 문 안에 있는데도 정북으로 가면 **오른쪽 문설주를 스침** — 회피가 할 일이 바로 그것임

F. 실습 6 — 파이썬 회피만 먼저 (15분)

- 이음새를 붙이기 전에, 회피 쪽이 혼자 도는지 확인함. Simulink 는 아직 띄우지 않음
- 터미널 1 에 Gazebo 가 떠 있는 상태에서 터미널 2 에

```
cd ~/Capstone-Design/1_2026-2학기_강의자료/10-주차별-강의자료/W09_vrx_lite
GOAL="-532 312" DEMO_THRUST=1 bash run_ca_nodes.sh
```

환경변수	하는 일
`GOAL="동 북"`	Simulink 대신 목표점을 1 Hz 로 박아 둠 (ENU)
`DEMO_THRUST=1`	좌·우 추진기에 600 N 을 10 Hz 로 밀어 배를 전진시킴

- `-532 312` 라는 숫자의 뜻 — 배는 동 -532 · 북 200 에서 뜸. 북 233 의 벽에 열린 문이 동 $-559 \sim -531$ 임. 그 문을 지나 **정북 112 m** 를 가리키는 점임
- 정상 출력 — 세 노드가 1초 간격으로 올라오고, 로그가 `/tmp/w09_ca_nodes.log` 에 쌓임

```
tail -f /tmp/w09_ca_nodes.log
```

로그에 보이는 줄	뜻
`OA Mode: OFF (atan2)`	통로가 열려 있음 — 목표 쪽으로 직진
`OA Mode: ON (Route 14, 1.1 m/s)`	호 다발에서 14번을 골랐음
`OA Mode: ON (All Blocked)`	전부 막혔음 — 탈출 중
`OA Mode: OFF (Goal Reached)`	목표 2 m 안으로 들어왔음

⚠ **`run_ca_nodes.sh` 가 띄우지 않는 것 둘이 이 주차의 요점임**

- `heading_controller_enu` — 헤딩 제어는 **Simulink** 가 함 (5주차 PD 를 그대로 씀)
- `goal_publisher_enu` — 목표점은 **Simulink** 의 웨이포인트 관리자가 냄
- 연구실 원본 런치(`vrx_avoidance_enu.launch.py`)는 이 둘까지 띄워서 제어까지 파이썬이 했음
- 이 주차는 그 둘을 빼고 **그 자리에 Simulink 를 끼움**. 그래서 이 실습에서는 둘을 환경변수로 **흉내만** 냄

✕ 실습이 끝나면 반드시 정리할 것

- 남은 `gz sim` 프로세스가 메모리를 수십 GB 까지 먹어 WSL 이 죽음

```
bash ../W03_vrx_lite/stop_vrx.sh
```

G. 실습 7 — 이음새를 붙여 돌림 (35분)

G-1. 네 걸음

① MATLAB — 설정을 올림

```
cd('\\wsl.localhost\Ubuntu-22.04\home\<사용자명>\Capstone-Design\1_2026-2학기_강의자료\10-주차별-강의자료\W09_simulink')
W09_setup
```

- 올리는 것 — 이득, 웨이포인트, 장애물 표, 경로 원점, `oa_timeout`

변수	값	어디서 쓰임
`Kp_psi` · `Kd_psi`	$400 \cdot 200$	헤딩 P-D (5주차 그대로)
`Kp_u` · `Ki_u`	$300 \cdot 100$	속도 PI (5주차 그대로)
`u_ref`	1.5 m/s	순항 속력
`Delta` · `R_LOS`	$10 \cdot 5$	LOS 유도 (5주차 그대로)
`oa_timeout`	0.5 s	**이음새** — 명령이 이보다 오래되면 LOS 로 되돌아감
`Ts_ctrl`	0.05 s	제어 주기

② 터미널 1 — Gazebo (실습 A 와 같음)

```
bash ../W03_vrx_lite/stop_vrx.sh
bash run_vrx_ca.sh
```

③ 터미널 2 — 회피 쪽 노드 셋 (이번에는 환경변수 **없이**)

```
bash run_ca_nodes.sh
```

④ MATLAB — 돌림

```
W09_vrx_run
```

G-2. `W09_vrx_run` 이 하는 다섯 걸음

걸음	하는 일	왜
0	ROS 네트워크 설정에서 `ROS_DOMAIN_ID` 를 읽어 맞춤	도메인이 다르면 토픽이 안 보임
1	토픽 **넷**을 점검. 하나라도 없으면 멈추고 조치를 찍음	반쯤 돌다 이상해지는 것보다 나음
2	RTF 를 20초 재고, 배가 뜬 자리를 읽어 경로 원점을 맞춤	아래 경고 상자
3	페이싱을 **측정한 RTF 로** 맞추고 `sim` 을 돎	모의가 실제 시간과 같은 속도로 가야 토픽이 안 밀림
4	`W09_plot` 으로 그림을 그리고 저장	보고서용
5	이음새 판정을 찍음	아래

> ⚠ **배가 뜬 자리와 경로 원점이 어긋나면 엉뚱한 곳으로 감**
>
> - 경로는 원점이 출발점인 좌표로 쓰여 있고, 월드는 원점이 따로 있음. `W09_vrx_run` 이 `path_north0` · `path_east0` 을 읽어 맞춤
> - 배가 뜬 자리가 그 원점에서 2 m 넘게 떨어져 있으면 **경고**를 찍음 — 월드를 바꿨거나 `W09_setup` 을 안 돌린 것임

> ⚠ **문서의 수치는 전부 실행 결과임**
> - 페이싱 비율은 고정값이 아니라 **그 자리에서 잰 RTF** 를 그대로 씀
> - RTF 가 0.25 아래로 나오면 `ultralite` 판을 쓰라고 찍어 줌

G-3. 끝나면 무엇을 보는가

- 화면에 찍히는 판정 둘

판정	뜻	이 월드에서 기대하는 값
`oa_n`	회피가 **켜진 횟수** (상승에지만 셈)	**2** — $x_n = 33$ 의 벽과 $x_n = 93$ 의 슬라럼
`clear_min`	월드 참값 장애물까지의 **최소 거리**	선체 반폭 1.23 m 보다 커야 함

- 그림에서 볼 것 셋

볼 것	어디
`psi_los` 와 `psi_ref` 가 **갈라지는 구간**	그 구간이 곧 회피 구간임
`oa` 신호의 네모 펄스 둘	위 `oa_n` 과 개수가 맞아야 함
궤적이 참값 장애물을 **비켜 가는지**	겹쳐 그린 붉은 점과 대조

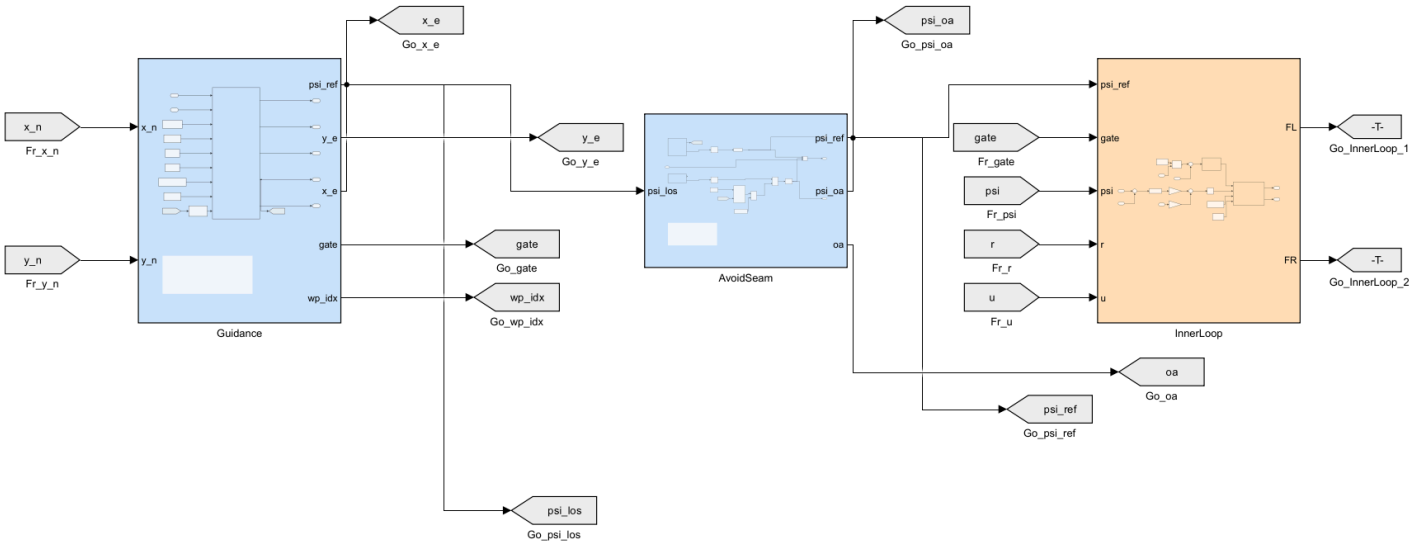
> ★ `oa_n` **이 4 로 나오면 세는 법이 틀린 것임**
> - 켜짐·꺼짐을 모두 세면 한 번 피할 때마다 2가 더해져 4가 됨
> - `W09_plot.m` 은 **상승에지만** 셈 — `sum(diff([0; on]) > 0)`
> - 본인이 다시 짤 때 흔히 걸리는 지점임

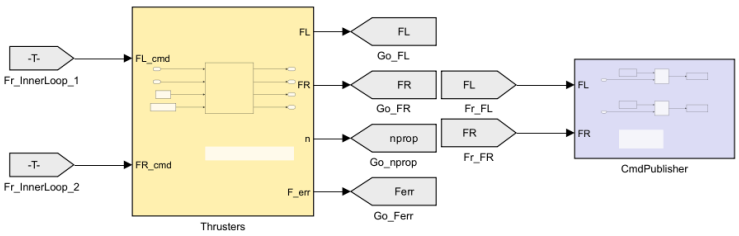
이럴 때	원인	조치
`oa_n` 이 0	회피 노드가 안 떠 있거나 목표점이 안 감	터미널 2 로그 · `ros2 topic echo /usv/state/goal_enu --once`
회피는 켜지는데 배가 안 꺾임	이음새의 스위치 문턱·포트 순서	`AvoidSeam` 을 열어 `Pick` 의 2번 입력을 확인
배가 반대로 꺾임	ENU↔NED 부호	`EnuToNed` 식을 확인 (1-7 절)
중간에 LOS 로 되돌아감	명령이 0.5 s 넘게 끊겼음	터미널 2 에서 `ros2 topic hz /vfh/command`

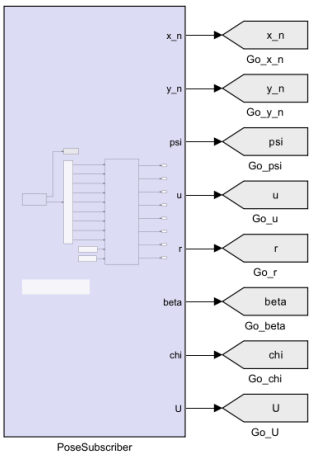
W09 실습 D — LOS 와 파이썬 회피를 한 제어기에 (Gazebo 필요)
Guidance -> AvoidSeam -> Inner Loop -> Thrusters -> Gazebo -> Navigation

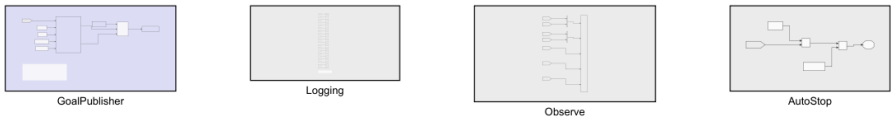
제어는 5주차 그대로다. 헤딩 P-D 와 속도 PI 는 블록 하나도 바뀌지 않았다.
바뀐 것은 **제어기에 들어가는 명령을 누가 내는가** 하나뿐이다.
oa_flag 가 0 : LOS 가 낸 psi_los 가 그대로 간다 (5주차와 완전히 같다)
oa_flag 가 1 : 파이썬이 낸 psi_oa 가 대신 간다
로그의 psi_los 와 psi_ref 가 갈라지는 구간이 회피 구간이다.

실행 전: (1) W09_setup (2) bash run_vrx_ca.sh (3) bash run_ca_nodes.sh









요소	의미
왼쪽 파랑	유도 — LOS 와 `AvoidSeam`
가운데 주황	제어 — 헤딩 P-D · 속도 PI. **5주차 그대로**
오른쪽 노랑	추력배분
연보라	ROS 입출력 — 명령 발행 · 자세 구독 · `GoalPublisher`
맨 오른쪽	로깅과 관찰

H. 실습 8 — 극좌표 그림 읽기 (10분)

- `run_ca_nodes.sh` 가 이미 `ca_viz.py` 를 띄웠음. 창이 둘로 나뉘어 있음

창	담는 것
왼쪽	NED 지도 — 지나온 항적 · 배 · 목표점. 위가 북(x_n), 오른쪽이 동(y_n)
오른쪽	극좌표 — **배 기준**. 위가 선수

- 오른쪽에 겹쳐 그려지는 선 넷과 읽는 법은 1-9 절의 표와 같음
- 제목 줄도 봄

제목	뜻
`LOS - Simulink drives (nearest X m)`	평상시
`AVOIDING - Python drives (nearest X m)`	회피 중 (붉은 글씨)

- 창 없이 그림만 저장하려면

```
VIZ_SAVE=/tmp/w09.png VIZ_SECS=25 bash run_ca_nodes.sh
```

> **그림의 글자가 영어인 것은 일부러임**
> - WSL 에 한글 폰트가 없으면 `matplotlib` 이 네모로 그림. 실시간 창은 영어로 둠
> - 보고서에 쓸 한글 그림은 MATLAB `W09_plot.m` 이 그림

I. 모델을 다시 만들어야 할 때

- 모델은 **손으로 그리지 않음**. 스크립트가 만듦

```
W09_setup
build_w09_models
```

- 하는 일 — `W05_1_vrx.slx` 를 복사 → 가지 하나를 끊음 → `AvoidSeam` 을 끼움 → `GoalPublisher` 를 붙임 → 배치를 정리하고 선 검사를 돌림
- 선 검사(`check_lines`)는 일곱 항목이 **모두 0** 이어야 통과임. 통과하면 아무것도 안 찍힘
- 즉 제어기는 다시 만들어지지 않음 — **복사된 같은 블록**임 (1-7 절)

마무리

이번 주차 요약

순서	한 일	확인 방법
1	6주차 선수추진기 배에 **3D 라이다**를 달았음	`make_wamv_ca.sh` 가 찍는 `x=0.700 y=0.000 z=1.900`
2	높이를 **실측으로** 정했음 (**1.90** m)	`probe_lidar_pose.sh` — **1.75** 까지 82 %, **1.80** 부터 0 %
3	라이다 세 판의 RTF 와 발행 주기를 쟀음	`measure_rtf.sh` — RTF 는 모두 **0.99**, `full` 만 **0.62** Hz
4	Rviz2 에서 장애물을 `PointCloud2` 로 확인했음	격자 위의 점들이 배와 함께 돎
5	다가갈수록 거리가 줄어드는 것을 숫자로 봤음	`check_obstacles.sh` 의 `가장가까운` 이 단조 감소
6	파이썬 회피기를 혼자 돌려 봤음	로그의 `OA Mode: ON (Route i, …)`
7	Simulink 에 **이음새**를 붙였음	`psi_los` 와 `psi_ref` 가 두 번 갈라짐
8	극좌표 그림으로 넷을 겹쳐 봤음	검정 선이 초록 또는 자홍 **위에** 얹힘

수업 진도 체크

★ 다음 주 수업을 따라가기 위한 최소 조건

필수 — 다음 주차가 이것 위에서 시작함

- [ ] `run_vrx_ca.sh` 로 라이다를 단 배가 뜨고, 라이다 토픽 둘이 보임
- [ ] Rviz2 에서 `PointCloud2` 가 보이고, Fixed Frame 을 왜 `wamv/wamv/base_link` 로 두는지 말할 수 있음
- [ ] `measure_rtf.sh` 를 돌려 본인 노트북의 표를 가지고 있음
- [ ] `W09_vrx_run` 이 끝까지 돌고 그림이 나옴

권장 — 해 두면 과제가 쉬움

- [ ] `probe_lidar.sh` 로 거리값 분포를 보고, 몰린 구간이 없음을 확인
- [ ] `check_obstacles.sh` 로 가장 가까운 거리가 줄어드는 것을 확인
- [ ] `AvoidSeam` 을 열어 `Pick` 의 세 입력이 각각 무엇인지 설명할 수 있음

심화 — 과제 ③ 과 Term Project 로 이어짐

- [ ] `plan.sim_time_sec` 을 바꿔 보고 궤적이 어떻게 달라지는지 봄
- [ ] 직사각형 선체와 원 근사를 바꿔 가며 통과 여부를 비교
- [ ] 본인 팀 미션에 회피 상태를 어디에 끼울지 그려 봄

과제 9 — 회피기를 흔들어 보고 숫자로 말하기

- **제출 기한**: 다음 주차 수업 시작 전
- **제출**: 보고서 1부 (PDF) + 로그·그림 파일

① 요구사항 — 돌리고 바꾸기

- 기본 설정(`lite` 판)으로 `W09_vrx_run` 을 한 번 돌려 **기준 결과**를 만듦
- 아래 셋 중 **둘 이상**을 바꿔 다시 돌림

바꿀 것	어디	무엇이 달라질 것으로 예상되는가
`plan.sim_time_sec` ($3 \rightarrow 5$ 또는 2)	회피 노드 매개변수	더 일찍 피하는가, 좁은 문을 못 지나가는가
`plan.clearance_weight` ($0.3 \rightarrow 1.0$)	같음	넓은 쪽으로 크게 돌아가는가
라이다 판 (`lite` → `ultralite`)	`run_vrx_ca.sh` 인수	발행 주기가 절반이면 회피가 늦는가

② 검증 — 필수

- 아래 넷을 **숫자로** 제시함. 말로만 쓰면 점수가 없음

항목	어디서
RTF 와 발행 주기	`measure_rtf.sh` 표 (본인 노트북 값)
`oa_n` (회피가 켜진 횟수)	`W09_vrx_run` 출력
`clear_min` (장애물까지 최소 거리)	같음
전체 주행 시간	같음

- 기준 결과와 바꾼 결과를 **같은 표에 나란히** 둘 것

③ 분석 — 5~10줄

- 아래 넷 중 **둘 이상**에 답함
  - `oa_flag` 가 거리 문턱이 아니라 통로 판정이라는 것을, 본인 로그의 한 순간을 들어 설명할 것
  - `sim_time_sec` 을 늘렸을 때 좁은 문(28 m)을 지나는 데 유리했는가 불리했는가. 왜 그런가
  - 발행 주기가 절반이 되었을 때 무엇이 먼저 나빠졌는가 — 회피 시점인가, 조타의 떨림인가
  - 선체를 원으로 근사하면 이 월드에서 어떤 구간이 통과 불가로 바뀌는가

④ 평가 기준

항목	배점
동작 — 기준 결과와 바꾼 결과가 둘 다 끝까지 돎	30 %
검증 수행 및 수치 제시	**40 %**
분석의 정확성	20 %
코드·설정 가독성 (무엇을 바꿨는지 분명한가)	10 %

막혔을 때

증상	가장 흔한 원인	조치
토픽이 하나도 안 보임	비대화형 셸이 `.bashrc` 를 건너뜀	`export ROS_DOMAIN_ID=7` 를 **명령 안에** 씀
Gazebo 창이 안 뜸	WSL 그래픽	PowerShell 에서 `wsl --shutdown` 뒤 다시
배가 추력을 줘도 안 움직임	`/vrx/release`	`gz topic -t /vrx/release -m gz.msgs.Empty -p ''`
라이다 점이 하나도 안 들어옴	URDF 를 안 만들었음	`bash make_wamv_ca.sh` 뒤 다시 띄움
가만히 있는데 회피가 켜져 있음	라이다가 **자기 배**를 봄	높이 확인 (1-2 절) · `probe_lidar.sh`
배가 아예 안 움직임, 오류는 없음	`full` 판 — 센서 끊김으로 판정됨	`lite` 로 (1-3 절)
Rviz 가 전부 `Status: Error`	Fixed Frame 이 `map`	`wamv/wamv/base_link` 로
`W09_vrx_run` 이 토픽 점검에서 멈춤	터미널 1·2 중 하나가 안 떠 있음	찍힌 조치 줄을 따라감
회피는 켜지는데 조타가 안 됨	이음새 스위치	`AvoidSeam` → `Pick` 입력 순서 (1-7 절)
MATLAB 이 느려지고 토픽이 밀림	페이싱이 RTF 와 안 맞음	`W09_vrx_run` 을 다시 (RTF 를 다시 잼)
WSL 이 갑자기 죽음	남은 `gz sim` 이 메모리를 먹음	끝낼 때마다 `stop_vrx.sh`

참고 자료

이 주차의 원본

- 회피 알고리즘 — `collision_avoidance_enu.py` (연구실 `vrx_control` 패키지). 이 문서의 1-4 ~ 1-6 절이 이 파일을 따라감
- 원본 런치 `vrx_avoidance_enu.launch.py` — 제어까지 파이썬이 하던 판. 이 주차는 그중 둘을 빼고 Simulink 를 끼움
- 실제 주행 영상 — https://youtu.be/KqoTraTfRd0

교수 강의자료 — 이 주차가 전제하는 쪽

- 센서신호처리 및 융합 — 센서 모델과 잡음, 전처리 (위 참조 강의 5번)
- 제어공학특론 — 좌표계와 회전 (위 참조 강의 4번)

볼트 내 문서

- 3주차-VRX-구축과-좌표계 — ENU · NED · `stop_vrx.sh`
- 5주차-웨이포인트-유도 — LOS 와 헤딩 P-D. 이번 주차가 **그대로 가져다 쓴** 제어기
- 6주차-선수추진기와-접안 — 이번 주차 배의 추진기 층
- 평가와-팀운영 · Term-Project-명세 — 중간 발표

배포 파일

- `W09_vrx_lite/` — 스크립트 11개 (위 "무엇을 돌리는가" 표)
- `W09_simulink/` — `W09_1_vrx.slx` · `W09_setup.m` · `W09_vrx_run.m` · `W09_plot.m` · `build_w09_models.m`

다음 주 예고

- **라이다 신호처리** — 이번 주차에 띄운 `/points` 를 직접 전처리하고 **부표를 탐지**함
- 이번 주차의 회피기는 `/scan` **한 고리**만 썼음. 열여섯 고리를 다 쓰면 무엇이 달라지는지 봄
- 군집(clustering)을 직접 짬 — 이번 주차에는 `cluster_min_points` · `cluster_max_gap_m` 두 매개변수로 **쓰기만** 했음
- 이번 주차에 만든 회피기는 그 뒤로도 계속 쓰임. 팀 미션 매니저에 **회피 상태**를 끼워 넣는 일이 Term Project 의 한 조각임